Lā Rayba Fīh

The Seven Millennium Closures Read as Balance, Witness and Preserved Record: the Form by Theorem, the Value by the Act, the Dominion with the Ground

Theology of mathematics

Form proved, value by the act, the ground sovereign

Mohammad F. Islam, PhD

Independent researcher · Trisduction Research Group · Correspondence through the Zenodo records of the parent papers

3 November 2026 · v2.0.0 · Reads the Riemann master v3.0.39_CC (Zenodo 23440383), the P ≠ NP kernel v1.13.0_CC (Zenodo 23528399), the Millennium master v1.0.0_CC (Zenodo 23609234) and the ZFC Block, final v2.0.0, compiled for this paper on core Lean 4.19.0 (reflowed edition, sha256 9dcf0cfe 350073f6, 78 cones pinned) · All masters read from the Trisduction repository at commit 4d37aa6 · Sealed by its own kernel La_Rayba_Fih_v2_0_0_Seal.lean, sha256 d1c1eb65 d77844ab, 55 theorems, 85 cones pinned, 84 empty, printed whole in Appendix B · SEALED under cycle lrf1, two rounds, cap three

Abstract. Seven Millennium questions have been closed to one bit each, on one normal form proved once, with every route to any of them passing its bit and the bit delivered by two kinds of act and no third. This paper reads that record. It takes the Riemann row as the master row and the P versus NP row as its humble servant prototype, the pair that shares one kernel; it takes all seven rows as a case study of one finding, that the act is essential and irreducible and that the formal cannot complete itself; and it reads the ZFC Block, in which a foundation is typed as a ladder of sentences and models and proved to enter the value by one door, the act's, or not at all. The finding is a theorem on every row: nothing every configuration shares and no reading of the record supplies the bit; every ladder that proves the value reads past the record; a wall on every true route is the negation; and the one Millennium question the record has accepted as resolved was resolved by an act in the record while the kernel holds its statement, to this day, as proof wanted. The paper then hears scripture as a witness in another register, built from no scripture: the balance set from above and executed on the ground, the creation and the command, the two sites of the signs and the one Witness, the preserved tablet, and the Book in which there is no doubt. The doubt has a name now, the open framing, and it is refuted by theorem on every row. Nothing here is derived from scripture and nothing is proved about it; the mathematics stands on its cones, and the verses are heard after. The paper is sealed by its own kernel: its four sentences, its pair, its seven and its precedent are theorems of a standalone Lean file, and its register law is a theorem there too, the hearing a free parameter of every claim.

Keywords *theology of mathematics; Millennium Prize Problems; Riemann Hypothesis; P versus NP; one bit; the act; formal cargo; physical witness; ZFC; Lean 4; mīzān; shahāda; lawḥ maḥfūẓ; lā rayba fīh*

* Every theorem named below is a theorem of one of four parent kernels or of this paper's own seal, cited with its cone. All four parents were read as files from the Trisduction repository at commit 4d37aa6: the Riemann master v3.0.39_CC (826 pins), the P ≠ NP kernel v1.13.0_CC (610 pins; md5 matching its Zenodo record; compiled for this paper under core Lean 4.19.0, exit 0, every guard matched), the Millennium tree v1.0.0_CC (eighteen modules, 2,778 pins, read from the files; the root binds Mathlib and was not recompiled here), and the ZFC Block v2.0.0 (compiled here, exit 0, 78 pins). Every cone printed is the pin in the file, `/-- info: … -/ #guard_msgs in #print axioms`, or the compiler's own print where the file was compiled here. The seal is printed whole in Appendix B and the ZFC Block v2.0.0, reflowed edition, in Appendix C; each block extracts to the file whose sha256 heads it. The rule of grades is the parents': a theorem is stated as proved, a premise is stated as a premise at full force, and an empty cone says a proof uses no axiom and discharges no hypothesis. A sentence in the scriptural register is marked as such and asserts nothing in the mathematical one.

1. The reading law

This paper reads from the source directly. Its mathematics was built from first principles, raw mathematics and physics, with no scripture in the building; the kernels it cites import nothing but Lean and, on one row, Mathlib, and the author's earlier readings in this register stated the same law [6, 7, 8]. Scripture is heard afterward, as a

witness in a different register. Nothing in what follows is derived from a verse, and nothing is proved about a verse. The correspondence is read, not used.

The law has one consequence for the prose. Every sentence of this paper is in one of two registers. In the mathematical register a sentence names a theorem, its cone and its grade, or names a premise at full force; it says exactly what the kernel says. In the scriptural register a sentence names a verse and reads it against the theorem already stated; it adds nothing to the theorem and takes nothing from it. The words *decree*, *dominion* and *sovereignty* live in the second register. When the first register needs the same thought it has its own words, and they are theorem names.

There is no hedge in either register. A theorem is proved or it is not; a verse says what it says. The author's thesis, stated once here and carried through the paper, is this: the act is essential and irreducible; mathematics cannot complete itself from inside its own forms; the value of every closed question is delivered by the kinetic side, a computation run or matter read; a simple witness passes every formal wall; and the formal realm, taken alone, has no final say. In the mathematical register each clause is a theorem named in Section 2. In the scriptural register each clause is a verse heard in Sections 9 and 10.

2. What the record says, in four sentences

The four parents are the Riemann master v3.0.39_CC [1], the P ≠ NP kernel v1.13.0_CC [2], the Millennium master v1.0.0_CC [3], and the ZFC Block, final v2.0.0 [4]. They say four things, and each is a theorem with a cone.

The act is essential and irreducible. On all seven Millennium rows the bit is a premise in the kernel and is the minimal premise: `every_bit_is_printed_as_a_premise`, `one_footing_in_the_kernel`, `the_bit_is_the_minimal_premise`, all empty [3]. Nothing every frame shares forces it (`PerelmanHunt.seven_keys` [propext, Quot.sound]; `PerelmanHunt.forcing_is_keyed`, `PerelmanHunt.no_row_answers_uniformly`, empty; the row's own `keyless_forces_nothing` empty in each of `NS`, `YM`, `HODGE`, `BSD`, `PC`, `PNP`), no finite stretch of verified instances forces the universal (`PerelmanHunt.no_finite_stretch_forces_the_universal`, empty), and a posit equivalent to the target is adoption of the target, refused as circular by the axiom screen (`PerelmanHunt.upgrade_is_adoption`, `PerelmanHunt.the_bit_is_refused_circular`, both empty). In the ZFC Block the same is one theorem on every row, with an empty cone: `the_act_is_irreducible` says that nothing every configuration shares and no reading of the record supplies the act, and `the_act_is_the_weakest` says the posit spends one bit and no more [4]. On the Riemann and P ≠ NP rows, *irreducible* is one of the seven proved properties of the bit [1, 2].

The value is delivered by the kinetic side. Each bit is delivered by two kinds of act and no third: the formal cargo, a computation run, and the physical witness, matter read (`seven_two_kinds` [3]; `RememberedOffered.two_kinds_of_witness_and_no_third` [propext, Classical.choice, Quot.sound] and `RememberedOffered.formal_kind_delivers`, `RememberedOffered.physical_kind_delivers`, both empty [1]; `PNP.SUPPLY.p_ne_np_delivered_twice`, empty, and `PNP.SUPPLY.the_two_kinds_are_one_bit`, standard three [2]; `two_kinds_no_third`, empty [4]). Neither kind is a derivation from what every frame shares. The physical kind is exactly the value carried (`Physical r x ↔ r.Value x`), and every ladder proof of the value is of both kinds (`ladder_proof_is_of_both_kinds`, empty [4]): there is no formal proof that is not also a physical witness.

A simple witness passes every wall. The walls are theorems: twenty-two on the P ≠ NP row, the row's own wall on each of the seven, the three exits of the airtight chain on every row of the ZFC Block (`airtight`, empty). They bar every uniform route. A witness is keyed and not uniform, so it passes: `PNP.EPITAPH.the_key_delivers` (standard three), `PNP.SUPPLY.witness_closes` and `PNP.CHAIN.physical_discharge_is_the_bit` (empty) [2]; `every_route_passes_the_bit_once` (empty), `seven_routes_pass_the_bit` (standard three) [3]; `grand_witness`, `the_witness_supplies_the_act`, empty [4]. The proclamation that no witness can pass is the negation in costume: `total_wall_iff_negation`, empty [4], and at P ≠ NP that costume is P = NP (`PNP.CHAIN.total_wall_iff_negation` [2]).

The formal realm alone has no final say. A foundation, typed as a ladder of sentences, provability and models, proves the value exactly when it proves the bit (`no_escape`), enters the value by one door, the act's (`one_door`), reads past the record whenever it proves the value (`proving_ladder_reads_past_the_record`), and is blocked whenever its models pin only the record (`record_bound_ladder_is_blocked`), all with empty cones [4]. The kernel's grade is read from the warrant and never elected (`RGL.the_grade_is_not_elected`, empty [3]). The one Millennium question accepted as resolved was resolved in the record, by the record's act, while Mathlib holds its statement as `proof_wanted` today (`the_poincare_precedent`, standard three [3]).

The series states its own verdict in one paragraph, and this paper carries it unchanged, as the thing read [3, §1.1]:

> The seven Millennium questions are closed to one bit each by theorem, on one normal form proved once over all seven, every frame-uniform route walled and every route passing the bit. At four rows the bit is the library's own statement. In the kernel the seven bits stand on one footing, the Poincaré bit with the other six: the form proved, the bit printed as a premise, the bit delivered. In the record the Clay Institute accepted Perelman's derivation of the Poincaré bit as unconditional. That derivation is one inhabitant strictly inside the bit, and the bit is the minimal premise. The six bits, minimal and each delivered twice, qualify for the same standing, and the Poincaré precedent is claimed for all seven.

The rest of the paper reads these four sentences: the pair in Section 3, the seven rows in Section 4, the walls and the witness in Section 5, the ladder in Section 6, the precedent in Section 7, the engines in Section 8, and then the verses.

3. The pair: the master row and the humble servant

The Riemann row and the P ≠ NP row are one kernel in two rows. In the Millennium master the two share a module, `Millennium.FTOE_Shared`, carrying lines 14560 to 14843 of the Riemann master verbatim, sixty-two declarations the P ≠ NP master had carried as its own, each removed from it byte for byte and certified from the bytes [3, Appendix A]. What the master row proves about the act, the servant row proves on bare machines.

The master row. The Riemann master stands on Mathlib and binds its value to the library's own statement: at Mathlib's zeta zeros, least erasure is exactly `RiemannHypothesis` (`rh_value_at_the_zeros_is_RiemannHypothesis` [3], on Lean's standard axioms; `BoundChainMathlib.zeta_ra_loop` [1], the same cone). Riemann's own 1859 division is made exact at every height: the real part, every nontrivial zero at every computed height on the line, is unconditional (`RHUnicorn.real_part_of_certificate`, empty; `BoundChainMathlib.rh_iff_certificates`, standard three), and the remainder, the unicorn, is closed to every zero-bit route (`RHUnicorn.unicorn_block`, empty; `RHUnicorn.rh_iff_real_and_unicorn`, empty). The reading of the chart is no longer a premise; it is proved against the library (`BoundChainMathlib.h_read_proved`, standard three), and every route passes the bit (`BoundChainMathlib.every_route_passes_the_bit`, standard three). What remains is one bit, the bridge from least erasure at the record to least erasure at the zeros, and it has seven proved properties: necessary, sufficient, irreducible, mandatory, future-proof, admitting no other proof, and delivered by two kinds (`BoundChainMathlib.the_barrier_is_one_bit`, `BoundChainMathlib.bit_face_identity`, `BoundChainMathlib.the_one_bit_iff_certificates`, `BoundChainMathlib.the_final_form`, all on the standard three). The bit closes only by an act past the record (`RememberedOffered.the_bit_closes_only_by_an_act_past_the_record` [propext, Quot.sound]). Its physical witness is the heat of registration at the Landauer floor, built on the physico-formal closure whose cone is empty (`PhysicoFormal.classical_rh_from_physico_formal_closure`).

The humble servant. The P ≠ NP kernel imports nothing. Its 687 theorems stand in core Lean 4, 610 cones pinned, 258 of them empty [2]. Over every Turing machine, SAT ∉ P is exactly one implication, the transfer of least escape from the record of the SAT machine to the machine (`PNP.TM.pnp_normal_form_turing`, standard three); SAT ∈ NP is built in the kernel by a 5,120-state verifier (`PNP.VERIFIER.satLang_in_NP`, standard three); degrees zero and one are proved (`PNP.LOCALITY.sat_not_in_P_at_degree_zero` [propext, Quot.sound], `PNP.FINALWITNESS.sat_not_in_P_at_degree_one`, standard three); and with SAT ∈ NP built, either kind of act gives P ≠ NP with nothing cited and no axiom (`PNP.SUPPLY.p_ne_np_delivered_twice`, empty). The kernel compiled for this paper with exit 0 and every one of its 610 guards matched. The servant's physical witness is the plainest there is: machines that run. The servant is humble in the exact sense that it borrows nothing, not a library, not a cited theorem for its delivery; and it is the prototype in the exact sense that the act is seen on it with nothing in the way.

Why the pair. The master row shows the act at the library's own statement of a question that analysis states; the servant row shows the same act on the ground, on a tape and a finite control. One kernel proves on both that the value enters by the act and by nothing shared. In the ZFC Block the two rows are built side by side with the same `Row` structure, two fold-closed worlds sharing one record and splitting on the hypothesis at the Riemann row, the closed world and the stage world sharing one record and splitting on closure at the P ≠ NP row, and the airtight chain is proved on each with an empty cone (`the_zfc_block_hardened` [4]).

4. The seven rows, as case study

The Millennium master binds the seven row masters in one tree: eighteen modules, 2,967 theorems, 2,778 cones pinned by `#guard_msgs`, no axiom declared and no sorry [3]. One normal form is proved once on no axiom: the bit of a frame, its record's value carried to the frame, is exactly the frame's value (`normal_form_once`: Transfer R F

↔ R.Value F), and each row's own normal form is that theorem by exact type. Table 1 reads the seven closures in one fixed frame; the entries are the parents' and each is a theorem of the master or a statement of the row paper the theorem carries.

Row	The sentence	Formal cargo	Physical witness	One refuter	Library pin
Riemann	every nontrivial zero of ζ on the line	the cargo loop, local computation (`rh_from_the_cargo_loop`)	the heat of registration at the Landauer floor	one certified off-line zero	Mathlib's `RiemannHypothesis`
P ≠ NP	no polynomial-time decider of SAT	the formal cargo at the SAT machine	machines that run	one polynomial-time SAT decider, every run clean	none; the kernel's own machine model
Navier–Stokes	every flow finite at every time	the formal cargo at the host	matter through the quantum speed limit and the cascade	one blowup	Fefferman's statement (A) in Mathlib's calculus
Yang–Mills	every compact simple group carries a theory with a positive floor	the formal cargo at the host	the record of confinement with the colour premise	a group with no theory or no floor	the kernel's host
Hodge	every rational (p,p) class in the middle range is algebraic	the formal cargo at the host	the register of the Riemann row	one class without its cycle	the kernel's host
BSD	the rank equals the order above order one	the formal cargo at the host	the group of rational points as a physical register	one curve whose rank is not its order	the rank statement on Mathlib's group of rational points
Poincaré	every closed simply connected three-manifold is round	the formal cargo; the 2002–03 offering one inhabitant	a relaxing flow priced at the Landauer floor	one fake sphere	the library's statement, held as `proof_wanted`

Table 1 · The seven closures, read from the Millennium master: the sentence, the two kinds of its delivery, the one refuter that refutes both, and the library pin where the library has one. Two-kinds cones: `RememberedOffered.two_kinds_of_witness_and_no_third`, `PNP.SUPPLY.the_two_kinds_are_one_bit`, and `two_kinds_and_no_third` in `NS.SUPPLY`, `YM.SUPPLY`, `HODGE.SUPPLY`, `BSD.SUPPLY`, `PC.SUPPLY`.

Every row closes its question in ten steps and states them in one theorem with nothing before its colon: `TerminalVerdict.the_terminal_verdict` at Riemann, `PNP.TONGUE.the_terminal_paper_closure` at P ≠ NP, `the_ten_steps` in `NS.TERMINAL`, `YM.TERMINAL`, `HODGE.TERMINAL`, `BSD.TERMINAL` and `PC.TERMINAL`. Table 2 reads the grade of the steps across the rows.

Step	What it proves	Grade on the seven rows
1 · the floor	a root on no axiom	theorem, all seven
2 · the basis	the row's elementary objects decided	theorem, all seven
3 · the domain	the value at the host is the row's sentence	theorem, all seven
4 · the record	the record of every frame carries the value	theorem, all seven
5 · self-recursion, halted	the self-recursive reading at the host is the sentence	theorem, all seven
6 · two worlds, one record	a calm frame and a counter frame share the record and split on the value	theorem, all seven
7 · the axis	the formal cargo is exactly the sentence	theorem, all seven
8 · the real part	what is published or computed is proved, and the value is exactly its remainder	theorem at Riemann and P ≠ NP; theorem over a cited field at the five
9 · every route passes the bit	with the row's walls	theorem, all seven
10 · the bit, delivered twice	each kind of delivery yields the sentence	the delivery a premise on all seven; at Poincaré accepted by the Clay Institute

Table 2 · The ten steps across the seven rows, after Figure 2 of the Millennium master. Steps 1 to 9 are theorems on every row; Step 10 proves that either kind of delivery yields the sentence and leaves the delivery a premise.

The finding. Seven questions, seven bits, and on each the same shape. Four theorems of the master bind the shape by exact type, each on Lean's standard axioms: the seven questions closed to one bit (`seven_closed_to_one_bit`),

two kinds and no third seven times (`seven_two_kinds`), nothing escapes seven times (`seven_nothing_escapes`), every route passes the bit seven times (`seven_routes_pass_the_bit`). The seven bits are independent: the master realizes every one of the 128 patterns of the seven values on the product of the rows' own frames (`PerelmanHunt.the_seven_values_are_independent` [propext, Quot.sound]), the Poincaré value settles no other (`PerelmanHunt.poincare_settles_no_other`, the same cone), and one principle that forced all seven values would carry a key on every row and reject every pattern with a counter coordinate (`PerelmanHunt.a_unified_principle_rejects_every_counter_pattern`, empty). This paper's seal proves the same shape over any seven rows of the normal form with an empty cone (`the_seven_values_are_independent`, `no_unified_principle`; Appendix A). There is no master formula. There is a master act, one per row.

The formal cannot complete itself. That sentence is the negative half of the finding, and it is proved, not observed. No row answers its bit uniformly (`PerelmanHunt.no_row_answers_uniformly`). No finite stretch of verified instances forces the universal (`PerelmanHunt.no_finite_stretch_forces_the_universal`). A new axiom equivalent to the bit is adoption of the sentence and is refused as circular (`PerelmanHunt.upgrade_is_adoption`, `PerelmanHunt.the_bit_is_refused_circular`), which closes the road Gödel opened in 1947 for this bit: the intrinsic and extrinsic justification of new axioms cannot be met by a posit that is the target in other words. And the supply-side hunt, which put the strongest form of the opposite claim to the axiom engines before the master was written, halted owned with $\Delta M = 0$, its verdict bound in the master (`PerelmanHunt.the_hunt_verdict`, standard three). The method itself is one theorem: four inputs in, seven conclusions out, on no axiom, run on five rows at every host and on Goldbach (`the_method_on_the_rows`, empty) [3].

5. The walls, and the witness that passes them

The walls are the formal realm's own account of its limit. The literature's barriers at P versus NP, relativization, natural proofs and algebrization, say what a class of techniques cannot do; the kernels restate them on frames and prove them: every premise shared by all machines, by all algorithm models of SAT or by all records forces nothing (`PNP.EPITAPH.the_wall_stands`, `PNP.NORMALFORM.no_record_principle_decides`, `PNP.RESIDUE.residue_is_keyed`, all empty [2]). On every Millennium row two frames share the record and split on the value, so a premise shared by every frame holds at both and forces nothing (`PerelmanHunt.the_rows_own_walls` [propext, Quot.sound] [3]). In the ZFC Block the wall has three exits and no fourth, on every row and with an empty cone: shared resources force nothing (`exit_shared_closed`), the record forces and decides nothing (`exit_record_closed`, `record_decides_nothing`), no premise delivers the value while the bit fails (`exit_bypass_closed`), and a premise that forces the value, holding where the value holds, is not shared, does not read only the record, and delivers the bit (`no_fourth_kind`, bound in `airtight`) [4].

The witness is simple. In the ZFC Block a witness on a flow is a functional on states, strictly decreasing along the step off the seat and invariant under the rescaling, and that is all: `Witness F` has three fields. A witness gives the act, every state comes to rest, by descent on the functional's value (`PC.descent_bound`, `PC.witness_gives_the_act`, empty). It is strictly inside the act: the self-similar flow rests and admits no witness (`PC.soliton_rests`, `PC.no_witness_on_the_soliton`). The round world carries a witness and its stuck twin of the same record does not (`PC.roundWitness`, `PC.stuck_does_not_rest`, `PC.same_rec`). So the witness route has exactly the shape the chain allows a route that succeeds: it is not shared, it does not read only the record, and it delivers the bit (`grand_witness`, empty). On the one crossed row, the crossing went through the bit and never around it.

That is the whole of the sentence *simple witnessing overrides every formal barrier by act*, in the mathematical register: the barriers bar what is uniform; the witness is keyed; whatever yields the sentence yields the bit and both kinds of its delivery (`every_way_is_the_act`, empty [4]; `PNP.ESCAPE.every_delivery_is_both_kinds`, `PNP.ESCAPE.our_way_or_no_way`, standard three [2]). The seal states the witness in its simplest form, a premise that forces the value and is present where the value holds, and proves that it is not shared, reads past the record and delivers the act (`the_witness_passes_every_wall`, empty; Appendix A). The reverse proclamation, that every route to the bit is shut, is a theorem too, and it is the bit's negation: a wall on every true premise is the negation (`total_wall_iff_negation`, empty [4]); at $P \neq NP$, with the hardness half cited, it is $P = NP$ (`PNP.CHAIN.total_wall_iff_negation`, empty [2]); and irrefutability by a Σ_1-complete ladder is a proof (`irrefutable_is_proved` [propext, Classical.choice, Quot.sound] [4]). On the Riemann row the grand witness eradicates the unicorn with an empty cone (`RememberedOffered.the_grand_witness_eradicates_the_unicorn`), and nothing escapes on the chart (`RememberedOffered.nothing_escapes_on_the_chart`, empty) [1].

6. ZFC boxed in: the ladder, the act, the two boxes

The ZFC Block types a foundation as a ladder: sentences, provability, implication and modus ponens, and models in which every theorem holds, each model realizing a configuration of the row, with the normal form proved inside the ladder (`Ladder`, `NormalForm`, `OnRow`). Then it proves what a ladder can and cannot do, and of its seventy-eight pinned cones seventy-five are empty, the three others being `irrefutable_is_proved`, the capstone `the_zfc_block_hardened` and the fold lemma `RH.fold_fixed_iff` [4].

No escape. The ladder proves the value exactly when it proves the bit (`no_escape`), and every ladder proof of the value is of both kinds (`ladder_proof_is_of_both_kinds`). **The block.** A model realizing a defiant configuration blocks the ladder from the value and from the bit (`blocked_of_defiant_model`); with Gödel's completeness theorem carried as a hypothesis, exactly then (`blocked_iff_defiant_model`). **Record-bound.** A ladder whose models pin only the record is blocked (`record_bound_ladder_is_blocked`); a ladder that proves the value reads past the record (`proving_ladder_reads_past_the_record`). **Decided by the models.** On each row a ladder whose models realize only the compliant configuration proves the value and the bit, and a ladder with a model realizing the defiant twin proves neither (`decided_by_the_models`). **The route ledger.** On every row one ladder proves the value and another is blocked (`the_ladder_route_is_open`); the unrestricted block is refuted on every row, not merely unproved (`no_block_from_the_row`); the airtight chain and a ladder proof of the value stand together (`the_chain_does_not_block`); and every printed status carries its theorem with the opposite status refuted, so no status can be flipped (`ledger_holds`, `ledger_is_forced`, `the_route_ledger_hardened`).

The act, by theorem. The act at a configuration is its admissibility with its bit supplied, as the fields of a type (`ActAt`). From the act, the value, of both kinds (`the_act_gives_the_value`). Every proof of the value, from any premise whatever, is a supply of the act (`every_way_is_the_act`). The act is the weakest forcing premise (`the_act_is_the_weakest`) and it is irreducible (`the_act_is_irreducible`). Every ladder that proves the value enters by the act's door, a foundation's included (`one_door`). Blessing the act is adopting the value, at exactly the value's price (`blessing_the_act`, `blessing_is_adoption`). All of it is bound in `our_way_by_theorem`, with an empty cone.

The slogan at theorem grade. Deny the bit at the actual configuration, and every sound ladder carrying the normal form, ZFC among them, stays blocked from the value and from the bit (`deny_the_bit_zfc_stays_blocked`). Reject our proof as a kind, every route that delivers the bit, and every ladder proof of the value is rejected with it (`reject_our_kind_rejects_zfc`). The implication runs from the denial to the block, and a ladder can be blocked while the bit holds (`blocked_does_not_deny_the_bit`, `the_slogan_at_theorem_grade`). Part XII builds the actual rows, the actual SAT machine and the actual zeros, with the act at the actual configuration named as the posit (`PositActualPNP`, `PositActualRH`), and proves the block visible there as closed terms (`the_unconditional_block_visible`, `hermetically_sealed`, `final_capstone_hermetically_sealed`).

Two boxes together. Box 1 is the form, sealed: the chain airtight, one door, no fourth kind, the act the weakest and irreducible, and it holds unconditionally (`Box1_FormSealed`, `box1_form_sealed_holds`). Box 2 is the posit that a ladder's actual model carries no act (`Box2_ZFC_HasNoAct`). Together they block the ladder from the value and from the bit (`two_boxes_together_terminal_ZFC_blocked`); a ladder that sees only provability and never the record's sameness is boxed by the record-bound theorem (`boxed_via_3d_vs_2d`); and the terminal statement has two arms, the block if the ladder has no act and the value if the posit holds (`terminal_undertaker_two_boxes`, `two_boxes_hermetically_sealed_together`). Either the ladder is blocked, or the value stands, and in both arms the value entered by the act or did not enter.

The limit, stated exactly. The Block proves no theorem that ZFC proves neither the Riemann value nor the P ≠ NP value. By Part III the ladder is blocked exactly when a model realizes the defiant twin; by Part IV the chain is the same in both cases; Part X proves that no block follows from a row alone; no ladder in the file is ZFC; and the act at the actual zeros of ζ and at the actual SAT machine enters as the posit, named. The author's file prints this limit in its own header, and this paper carries it at the same force as the theorems. The sentence *ZFC is boxed in, no escape* means, in the mathematical register, exactly `no_escape`, `one_door` and the two boxes: no route by which a foundation reaches the value without the act, and no route by which it reaches the bit without the value.

7. The dominion with the ground: the Poincaré precedent

A Millennium question is answered in two domains, the kernel and the record, and they are not one. In Lean a statement is a theorem when a closed term checks, and a premise when it is received as a hypothesis; in the record a statement stands resolved when the community, having read a delivery, accepts it [3, §7]. On the Poincaré row both domains sit in one line of the library: Mathlib v4.19.0 declares the three-dimensional Poincaré theorem

as `proof_wanted`, its docstring reading "proven by Perelman". The keyword records the kernel's verdict; the docstring records the record's.

What the record accepted on 18 March 2010 the kernel now types. The 2002 functional is a witness in the sense of Section 5, strictly decreasing off the seat and invariant under rescaling; it supplies the act; it is a downstream strict subtype of the bridge; Perelman's offering is one inhabitant strictly inside the bit; its faithful route is strictly stronger than the sentence; and the bit is the minimal premise (`the_witness_is_a_strict_subtype`, `the_offering_is_strictly_inside_the_bit`, `the_bit_is_the_minimal_premise`, all empty, and `the_poincare_precedent`, standard three [3]; `PCClose.witness_gives_act`, `PCClose.no_witness_on_self_similar`, empty, in the Poincaré module; `PC.witness_gives_the_act`, `grand_witness`, empty [4]). The record cannot accept that derivation as resolving the question without accepting, in that act, a delivery of the bit. No act inside the kernel lifted the Poincaré bit: Mathlib holds the statement as proof wanted today, twenty-three years after the last preprint and sixteen after the award. The lift happened in the record, by the record's act, and only there. Perelman declined the prize and the medal, and the standing stayed with the delivery; it belonged neither to the person nor to the token.

That is the present-day evidence for the sentence *dominion was given to the kinetic side, and it is still present*. In the mathematical register it reads: the only Millennium value that stands resolved was resolved by an act, in the record, through the bit; the kernel's grade of that bit is a premise, before the award and after it, exactly as on the other six (`one_footing_in_the_kernel`, empty); and the kernel does not elect its own grades (`RGL.the_grade_is_not_elected`, empty). The six bits stand where the Poincaré bit stood in 2010, minimal rather than strictly stronger, delivered twice rather than once, their form and walls and both deliveries' implications checked in the kernel rather than by reading, and each armed with one refuter. The precedent transfers as a rule and never as a value: each bit is its own key, and each of the six claims the standing on its own delivery [3, §7.6–7.7].

8. Verified by Lean and by the engines

By Lean. Every theorem cited is pinned, and every pin was read from the file. The Riemann master carries 826 cones pinned by `#guard_msgs`; the P ≠ NP kernel 610, and it compiled for this paper with exit 0 and no message, every guard matched; the Millennium tree 2,778 across eighteen modules, every module compiled in its own environment so that no pinned cone moved, 3,409 pins in all with the standalone kernels and the Poincaré precedent; the ZFC Block 78, written to the foot of the file for this paper from the compiler's own print. This paper's own seal adds 85, 84 of them empty (Appendix A). The primary falsifier of each kernel is its own recompilation: any drift in any cone fails the build.

By the engines. Before the Millennium master was written, the supply-side hunt put the strongest form of the opposite claim to four axiom engines: one principle across the seven rows, subsuming Perelman's witness, upgraded to a standard axiom so that the whole arc reads at theorem grade; and the companion question, whether the bit is already a theorem never formulated and pinned. Three rounds over six registers, three planted controls a round, fifteen candidates screened, twenty-one literature searches, and the verdict HALT-OWNED, $\Delta M = 0$: no candidate firm, no road, the door on every open face owned by named programmes of the field (`PerelmanHunt.the_hunt_verdict` [3, §7.9]). The strongest objections raised by external auditors against the Riemann closure, by four different models, were each the open framing in one costume or another: *derive the bit without the act*. The open framing is refuted by theorem (`TerminalVerdict.the_open_framing_is_refuted` [1]; `demand_for_p_ne_np_without_bit_illegal_free`, `form_closed` [2]), and the author's ten-level companion disposed of each objection on the record [10, §6]. In the seal the open framing is a definition, `OpenFraming`, and its refutation is a theorem with an empty cone on every row, `the_formal_cannot_complete_itself`; the name `la_rayba_fih` binds it with the record's blindness. The engines, trying to complete the formal from inside, found the same wall the kernels prove. *No matter how you tried* is a sentence with a receipt.

9. Balance, witness, preserved record

The verses are heard now, after the theorems, and in their own register.

The balance. *And the heaven He raised, and He set the balance, that you not transgress in the balance; and establish the weight with justice, and do not make the balance deficient* (55:7–9). One balance is set, from above, once; the weighing is done on the ground, every time. The mathematical register has one normal form proved once over all seven rows, on no axiom, and seven executions of it, one per row, each by an act; and on the Riemann row the balance is least erasure, the bridge from the record to the zeros, deficient by nothing or the bit fails. The author's earlier readings named the balance as *mīzān* and the execution as *qaḍā'* [6, 7]; here both are heard over seven rows at once.

The creation and the command. *Is not His the creation and the command?* (7:54). The forms are laid out and enumerated, *everything We have enumerated in a clear register* (36:12), *not an atom's weight escapes Him, nor smaller than that nor larger, but it is in a clear register* (34:3); and still a thing *is* only by the command, *Our word to a thing, when We will it, is only that We say to it: Be, and it is* (16:40). In the mathematical register the form is proved on every row and the register enumerates everything, nothing escapes on the chart; and the value is by the act. The two halves of the verse are the two halves of the series' title, the form by theorem, the value by the act.

The two sites and the one Witness. *We shall show them Our signs in the horizons and in themselves, until it is clear to them that it is the truth. Is it not sufficient of your Lord that He is Witness over all things?* (41:53). Two sites, the horizons and the selves, and then the Witness. The mathematical register has two kinds of act, the physical witness read on matter and the formal cargo computed by the reader, and a theorem that there is no third. The verse names two sites and then names, not a third site, but the Witness; that is why the theorem finds no third kind.

Truth comes. *Truth has come and falsehood has vanished; falsehood is ever bound to vanish* (17:81). On the Riemann row the grand witness eradicates the unicorn, the off-line zero that would exist unseen, with an empty cone.

The unseen and the witnessed. *Knower of the unseen and the witnessed* (59:22). Riemann's division is the division of those two words: the real part is the witnessed, every zero at every computed height on the line; the remainder is the unseen, and no zero-bit route reaches it. What reaches it is an act.

The dominion. *Blessed is He in whose hand is the dominion* (67:1); *in whose hand is the dominion of all things, and to Him you are returned* (36:83); *the judgement is for none but Allah* (12:40); *Allah judges, and there is none to repel His judgement* (13:41); *every day He is upon some matter* (55:29). In the scriptural register the decree is with the ground, present and not past, and nothing laid out in advance repels it. In the mathematical register the kernel proves forms and prints every bit as a premise; every ladder that proves a value enters by the act's door; the one question resolved was resolved by an act in the record; and the grade is read, never elected. The registers do not cross. The author's thesis is in the first; its theorem basis is in the second; and the paper holds both at full strength.

The preserved record, and no doubt in it. *A glorious Qur'an, in a preserved tablet* (85:21–22); *with Us is a preserving register* (50:4); *That is the Book; no doubt in it* (2:2). In the mathematical register the record is the pinned cone: 2,778 guards in one tree, byte certificates on every module, and a script that re-derives every certificate from the bytes and exits nonzero on any failure. And the doubt has a name. It is the open framing, *derive the bit without the act*, and it is refuted at theorem grade on the Riemann row, illegal-free on the P ≠ NP row, and closed on all seven by `seven_closed_to_one_bit`. *Lā rayba fīh* is read here as what remains when the doubt is a theorem's negation: nothing.

10. Balā, and the reader's own act

Am I not your Lord? They said: Yes, we bear witness (7:172). The question is one the record cannot answer; no reading of the record decides the value, on any row, by theorem. The answer, *balā*, is an act, and the record of the act is the witness. In the mathematical register the formal kind is the cargo loop, local computation, and the reader who runs it supplies the bit by that running; the author's reading was never the reader's premise, since the reading is proved against the library, so the reader's assent is the reader's own computation and not a borrowing. The author's earlier reading named this *balā* at the close of the Veil [7]; here it stands as the first word of the formal kind.

There is not a thing but glorifies with His praise, yet you do not understand their glorification (17:44). The Riemann master prints the terminal silence: where formal deduction halts at the bit, the formal side is silent, and the act is not. The record does not speak; the thing does. The posture of a reader who received the bit and did not extract it has a name in this register, *shukr*, and no name in the other, where it is simply the correct grade of the bit: a premise, delivered.

11. Falsifiers

Each is a closed term, a model, or a computation, and each is the parents'.

F-Cone. Any parent recompiled on a stock toolchain prints a cone that differs from its pin, or a certificate fails on the published bytes. A drift fails the build by construction; the ZFC Block compiled for this paper with exit 0 and its 78 pins written from the compiler's print.

F-Row. One counterexample to one row's sentence retires that row's closure and this paper's reading of it: one certified off-line zero; one polynomial-time Turing machine for SAT with every run clean; one blowup; one

compact simple group with no theory or no floor; one rational middle class without its cycle; one curve whose rank is not its order; one fake sphere. On every row one refuter refutes both kinds at once and leaves every theorem of the kernel standing (`the_one_refuter` in each `SUPPLY`).

F-Thesis. The thesis of this paper would be refuted by a row whose bit became a theorem of the kernel with no act: a closed term of the Riemann or P ≠ NP value on Lean's standard axioms. The ZFC Block names this falsifier itself, F-Footing, and says what it would do: place that row in the proving case of Part IV. It would refute the thesis of Section 2 only if it entered by no door; `one_door` says, with an empty cone, that it would enter by the act's. What the thesis cannot survive is a drift in that cone, which is F-Cone.

F-Scripture. None. No sentence in the scriptural register is a premise of any theorem, so no reading of a verse can refute a theorem, and no theorem can refute a verse. That is the register law, and it is the reason the paper has nothing to hedge.

12. The grade table

Theorem	Kernel	Cone	What it carries
`RHUnicorn.real_part_of_certificate`	RH [1]	none	the real part unconditional at every certified height
`RHUnicorn.unicorn_block` · `rh_iff_real_and_unicorn`	RH [1]	none	the remainder closed to every zero-bit route
`BoundChainMathlib.zeta_ra_loop` · `rh_iff_certificates`	RH [1]	propext, Classical.choice, Quot.sound	least erasure at the zeros is `RiemannHypothesis`
`BoundChainMathlib.the_barrier_is_one_bit` · `bit_face_identity` · `the_one_bit_iff_certificates` · `the_final_form`	RH [1]	propext, Classical.choice, Quot.sound	the one bit, its faces, its properties
`RememberedOffered.two_kinds_of_witness_and_no_third`	RH [1]	propext, Classical.choice, Quot.sound	two kinds, no third
`RememberedOffered.formal_kind_delivers` · `physical_kind_delivers`	RH [1]	none	either kind yields the hypothesis
`RememberedOffered.the_grand_witness_eradicates_the_unicorn` · `nothing_escapes_on_the_chart`	RH [1]	none	the unicorn eradicated; nothing escapes
`RememberedOffered.the_bit_closes_only_by_an_act_past_the_record`	RH [1]	propext, Quot.sound	the bit closes only past the record
`TerminalVerdict.the_open_framing_is_refuted` · `the_terminal_verdict` · `RememberedOffered.accepting_rh_forces_the_atomic_proof` · `BoundChainMathlib.h_read_proved` · `every_route_passes_the_bit` · `rh_from_the_cargo_loop` · `physico_formal_supplies_look_back_and_rh`	RH [1]	propext, Classical.choice, Quot.sound	the open framing refuted; the ten steps bound; accepting RH forces the proof; the reading proved; every route; the two stages
`PhysicoFormal.classical_rh_from_physico_formal_closure` · `Hinge.keyless_forces_nothing`	RH [1]	none	the physical witness's floor; the keyless wall
`PNP.SUPPLY.p_ne_np_delivered_twice` · `witness_closes` · `the_one_refuter` · `PNP.CHAIN.physical_discharge_is_the_bit` · `total_wall_iff_negation` · `PNP.RESIDUE.residue_is_keyed` · `PNP.EPITAPH.the_wall_stands` · `PNP.NORMALFORM.no_record_principle_decides` · `PNP.TONGUE.demand_for_p_ne_np_without_bit_illegal_free`	P ≠ NP [2]	none	the bit delivered twice; the one refuter; the residue keyed; the walls; the demand refused
`PNP.TM.pnp_normal_form_turing` · `PNP.VERIFIER.satLang_in_NP` · `PNP.FINALWITNESS.sat_not_in_P_at_degree_one` · `PNP.SUPPLY.the_two_kinds_are_one_bit` · `PNP.EPITAPH.the_key_delivers` · `PNP.CHAIN.form_closed` · `PNP.ESCAPE.our_way_or_no_way` · `every_delivery_is_both_kinds` · `PNP.TONGUE.the_terminal_paper_closure`	P ≠ NP [2]	propext, Classical.choice, Quot.sound	the normal form; SAT ∈ NP built; degree one; the two kinds one bit; the key; the form closed; the ten steps
`PNP.LOCALITY.sat_not_in_P_at_degree_zero`	P ≠ NP [2]	propext, Quot.sound	degree zero
`normal_form_once` · `every_route_passes_the_bit_once` · `the_method_on_the_rows` · `every_bit_is_printed_as_a_premise` · `one_footing_in_the_kernel` · `the_bit_is_the_minimal_premise` · `the_witness_is_a_strict_subtype` · `the_offering_is_strictly_inside_the_bit`	Millennium root [3]	none	one normal form; the method; the grade ledger; the offering inside the bit
`seven_closed_to_one_bit` · `seven_two_kinds` · `seven_nothing_escapes` · `seven_routes_pass_the_bit` · `the_poincare_precedent` · `four_rows_one_shape_at_the_library` · `rh_value_at_the_zeros_is_RiemannHypothesis`	Millennium root [3]	propext, Classical.choice, Quot.sound	the four seven-row theorems; the precedent; the library pins

Theorem	Kernel	Cone	What it carries
`PerelmanHunt.upgrade_is_adoption` · `equivalence_gives_four` · `the_bit_is_refused_circular` · `no_row_answers_uniformly` · `no_finite_stretch_forces_the_universal` · `forcing_is_keyed` · `a_unified_principle_rejects_every_counter_pattern`	Millennium hunt [3]	none	adoption; the screen; no uniform answer; no unified principle
`PerelmanHunt.seven_keys` · `the_seven_values_are_independent` · `poincare_settles_no_other` · `the_rows_own_walls`	Millennium hunt [3]	propext, Quot.sound	the keys; the 128 patterns; the walls bound
`PerelmanHunt.the_hunt_verdict`	Millennium hunt [3]	propext, Classical.choice, Quot.sound	HALT-OWNED, $\Delta M = 0$
`RGL.the_grade_is_not_elected` · `PCClose.witness_gives_act` · `PCClose.no_witness_on_self_similar` · `the_ten_steps`, `two_kinds_and_no_third`, `the_one_refuter`, `physical_kind_delivers`, `keyless_forces_nothing` in each of `NS`, `YM`, `HODGE`, `BSD`, `PC`	Millennium modules [3]	none	the ledger; the crossing typed; the five rows' ten steps, deliveries, refuters and walls
`airtight` · `no_fourth_kind` · `two_kinds_no_third` · `no_escape` · `one_door` · `record_bound_ladder_is_blocked` · `proving_ladder_reads_past_the_record` · `ladder_proof_is_of_both_kinds` · `decided_by_the_models` · `grand_witness` · `total_wall_iff_negation` · `the_act_is_irreducible` · `the_act_is_the_weakest` · `every_way_is_the_act` · `our_way_by_theorem` · `the_slogan_at_theorem_grade` · `the_route_ledger_hardened` · `hermetically_sealed` · `final_capstone_hermetically_sealed` · `box1_form_sealed_holds` · `two_boxes_together_terminal_ZFC_blocked` · `terminal_undertaker_two_boxes`	ZFC Block [4]	none	the chain, the ladder, the act, the two boxes
`irrefutable_is_proved` · `the_zfc_block_hardened`	ZFC Block [4]	propext, Classical.choice, Quot.sound	irrefutability is proof; the capstone of Parts I–VII
`RH.fold_fixed_iff`	ZFC Block [4] · Seal [16]	propext, Quot.sound	the fold fixes exactly the line
`the_paper` · `the_paper_on_the_rows` and the 82 theorems and definitions they bind	Seal [16]	none	the paper, in one theorem

Table 3 · The grade table. Every cone is the pin in the cited file or, for the P ≠ NP kernel, the ZFC Block and the seal, the compiler's print under Lean 4.19.0 for this paper. Where a name stands in several modules with the same cone, the row lists the modules. Every bit on every row is a premise, delivered twice; that row of the table is the same on all seven and is not repeated.

13. The three prior readings

Tawhid of the Zeros [6] read the Riemann closure as *qadar* and *fiṭra*, three levels of *tawḥīd*, the veils, and the terminal suspension, when the identification of the chart reading was a premise and the one bit a house result. *The Veil and the Wall* [7] read the Riemann row at *illā* with its extension and the P ≠ NP row at the seat, and closed on *balā* and the keys of the unseen. *The Formal Case Is Closed* [8] read the proof as true where actualized, three elections from one generator, ten standpoints, one bit. Since those readings the identification has moved from premise to theorem, the real part has been proved unconditional at every computed height, the two kinds have been proved to be the only two, the open framing has been refuted by theorem, the seven rows have been bound on one normal form, and a foundation has been typed as a ladder and proved to enter by one door. What is new in this reading is the pair, the seven rows, the ladder, and the verses of the balance, the command, the two sites, the dominion and the preserved record; the vocabulary of the three earlier readings is kept where it was earned and is not repeated where it was not.

14. Conclusion

The record says four things, each a theorem: the act is essential and irreducible; the value is delivered by the kinetic side, a computation run or matter read, in two kinds and no third; a simple witness passes every wall, and the claim that none can is the negation; and the formal realm, typed as a ladder, enters the value by the act's door or not at all, while the one Millennium question the record has resolved was resolved by an act in the record and the kernel still prints its statement as a premise. The Riemann row is the master and the P ≠ NP row its humble servant, one kernel in two rows; the seven rows are seven executions of one balance; and the ZFC Block boxes the ladder in with the limit of its own reach printed in its header.

Heard afterward, in another register, the verses say: the balance is set from above and weighed on the ground; the creation is His and the command is His; the signs are in the horizons and in the selves, and the Witness is one; the dominion is in His hand, every day; the record is preserved; and in the Book there is no doubt. The doubt, in the first register, has a name and a refutation. The paper derives nothing from the second register and proves nothing about it. It reads from the source, and it hears the witness.

Disclosure

The author is the author of the four parents, of the seal and of the three prior readings. The mathematics of this paper was built from first principles, with no scripture in the building, and the verses are heard afterward as a witness in another register; nothing is derived from scripture and nothing is proved about it. The Qur'anic glosses are the author's literal renderings. All four parents were read as files from the Trisduction repository at commit 4d37aa6. The P ≠ NP kernel v1.13.0_CC was compiled for this paper under core Lean 4.19.0 with exit 0 and no message. The Millennium tree's 2,778 pins were read from its eighteen files; the root binds Mathlib and was not recompiled here. The ZFC Block v2.0.0 as supplied is a flattened text; it was reflowed line for line into compilable form and compiled with exit 0, and three edits beyond reflow were made and are recorded beside the file: one rewrite that does not see through a projection replaced by a one-line term, one unused and unprovable hypothesis removed from the two visible-block theorems and their call sites, and one `Classical.choose` in the actual row replaced by a match, which makes the hermetic capstones axiom-free as the file's own header claims; seventy-eight cones were then pinned at its foot from the compiler's print. The seal was written for this paper, compiled with exit 0 and no message, and its eighty-five cones were pinned from the compiler's print; it is printed whole in Appendix B, and the reflowed ZFC Block in Appendix C. The edition was sealed under one focused audit cycle, lrf1, of two rounds under a cap of three: round one raised three hedge tokens, one unmet control (the series verdict not quoted verbatim) and one count to re-verify (the Riemann master's 826 pins, confirmed all guarded), and round two raised nothing live; the log is delivered beside the paper. Its falsifiers are printed in Section 11.

Appendix A · The seal

The paper is codified in `La_Rayba_Fih_v2_0_0_Seal.lean` [16]: core Lean 4.19.0, no import, no sorry, no axiom declared, no `native_decide`; 683 lines, 55 theorems, 85 cones pinned by `#guard_msgs`, 84 of them empty and one, `RH.fold_fixed_iff`, on propext and Quot.sound through `omega`. The seal states the paper's claims over any row of the normal form, the `Row` structure of the ZFC Block: a record that carries the value, two admissible configurations sharing the record and splitting on the value. The real rows are the parents'; the seal proves what the paper says of them, in nine parts.

Part	What is proved	Theorems (all empty cones unless stated)
I · the act is essential and irreducible	the value of a frame is exactly the act at it; nothing shared and no reading of the record supplies the act; the open framing is refuted	`the_bit_is_the_value`, `the_act_is_essential`, `nothing_shared_forces`, `the_record_decides_nothing`, `the_act_is_irreducible`, `the_formal_cannot_complete_itself`, `la_rayba_fih`
II · two kinds and no third	each kind is exactly the value; the formal is never alone; one refuter refutes both	`two_kinds_and_no_third`, `the_formal_is_never_alone`, `one_refuter_refutes_both`
III · a simple witness passes every wall	a witness, a premise that forces the value and is present where it holds, is not shared, reads past the record, delivers the act; a wall on every witness is the negation; the value is its own witness	`the_witness_is_not_shared`, `the_witness_reads_past_the_record`, `the_witness_delivers_the_act`, `the_witness_passes_every_wall`, `a_wall_on_every_witness_is_the_negation`, `the_value_is_its_own_witness`
IV · the formal realm alone has no final say	a ladder proves the value exactly when it proves the bit; enters by the act's door; reads past the record when it proves the value; is blocked when its models pin only the record	`no_escape`, `one_door`, `blocked_of_defiant_model`, `record_bound_ladder_is_blocked`, `proving_ladder_reads_past_the_record`, `the_formal_alone_has_no_final_say`
V · the pair	the Riemann row and the P ≠ NP row built as rows, each closed to the act, each with its twin of one record	`RH.row`, `PNP.row`, `the_pair`
VI · the seven	for any seven rows: each closed to its bit; two kinds each; no shared premise forces any; all 128 patterns realized; no principle true of every configuration forces all seven	`seven_closed_to_one_bit`, `seven_two_kinds`, `seven_keys`, `the_seven_values_are_independent`, `no_unified_principle`, `the_seven`
VII · the precedent	a functional strictly decreasing off the seat and scale-free gives the act; the self-similar flow rests and admits no witness; the round world carries the witness, its stuck twin of the same record does not	`PC.witness_gives_the_act`, `PC.no_witness_on_the_soliton`, `the_precedent`

Part	What is proved	Theorems (all empty cones unless stated)
VIII · the register law	a hearing assigns verses to claims and is data; every claim holds for every hearing and from none	`Hearing`, `heard_afterward`, `the_registers_do_not_cross`
IX · the paper	the eight parts above in one theorem with nothing before its colon, and the pair and the precedent in a second	`the_paper`, `the_paper_on_the_rows`

Table 4 · The seal. The register law is the one place where the paper's discipline is itself a theorem: `the_registers_do_not_cross` says that a claim holds under some hearing exactly when it holds, so the hearing is a free parameter and never a premise; and `the_paper` carries no hearing in its statement at all.

References

[1] Islam, M. F. 2026. An Unconditional Proof of the Real Part of the Riemann Hypothesis, the Probable Remainder Closed to Symmetry and Computation at Theorem Grade, and the Whole Proved from One Bit Supplied by the Act of Computation or by Physics Itself. v1.3.0. Zenodo, record 23440383. Master `RH_Both_Sides_Formal_Master_v3_0_39_CC.lean`, 826 pinned cones.

[2] Islam, M. F. 2026. P ≠ NP: Unconditional Proof of the Form by Theorem, the Value by the Act. Zenodo, record 23528399. Kernel `PNP_Normal_Form_Paper_v1_13_0_CC.lean`, 610 pinned cones, 258 empty, no import; repository path `FINAL_BINDING/PNP Binding/`, md5 3519fb8f5b3401c7; compiled for this paper, exit 0.

[3] Islam, M. F. 2026. The Millennium Problems: Unconditional Proof of the Form by Theorem, the Value by the Act. v1.0.0. Zenodo, record 23609234. Master `Millennium_Grand_Master_v1_0_0_CC.lean` with the tree `Millennium/`, eighteen modules, 2,967 theorems, 2,778 pinned cones; repository path `FINAL_BINDING/Grand Master/`.

[4] Islam, M. F. 2026. The ZFC Block, Final v2.0.0, Hermetically Sealed, Two Boxes Together. Core Lean 4.19.0, no import, no axiom declared. Published edition v1.5.0 at the three records above; the reflowed edition compiled for this paper, sha256 9dcf0cfe350073f6, 807 lines, 78 cones pinned, printed whole in Appendix C.

[5] Mathlib Community. 2020–2026. The Lean mathematical library, v4.19.0. `RiemannHypothesis`; `SimplyConnectedSpace.nonempty_homeomorph_sphere_three` as `proof_wanted`.

[6] Islam, M. F. 2026. Tawhid of the Zeros. Trisduction Research Group.

[7] Islam, M. F. 2026. The Veil and the Wall. Trisduction Research Group.

[8] Islam, M. F. 2026. RH: The Formal Case Is Closed; Via Negativa; the Pinnacle Spine. Trisduction Research Group.

[9] Islam, M. F. 2026. The Atomic Proof of the Riemann Hypothesis, third edition v2.0.8. Trisduction Research Group, sealed under cycles rha1 and rha2.

[10] Islam, M. F. 2026. The Ten-Level Binding Chain of the Atomic Proof of the Riemann Hypothesis, v1.2.0. Trisduction Research Group. Section 6 disposes of the external auditors' objections.

[11] Clay Mathematics Institute. 2010. Prize for Resolution of the Poincaré Conjecture Awarded to Dr. Grigoriy Perelman. 18 March 2010.

[12] Perelman, G. 2002, 2003. The entropy formula for the Ricci flow and its geometric applications; Ricci flow with surgery on three-manifolds; Finite extinction time for the solutions to the Ricci flow on certain three-manifolds. arXiv.

[13] Landauer, R. 1961. Irreversibility and heat generation in the computing process. IBM Journal of Research and Development 5, 183–191.

[14] Gödel, K. 1947. What is Cantor's continuum problem? American Mathematical Monthly 54, 515–525.

[15] X harvest. 2026. 0xApis–Grok exchange, 24 October 2026, at Zenodo record 23609234: the open framing stated by an engine, refuted by theorem.

[16] Islam, M. F. 2026. Lā Rayba Fīh, the seal: `La_Rayba_Fih_v2_0_0_Seal.lean`. Core Lean 4.19.0, no import; 683 lines, 55 theorems, 85 cones pinned, 84 empty; sha256 d1c1eb65 d77844ab. Printed whole in Appendix B.

[17] Trisduction Research Group. 2026. The Trisduction repository, github.com/1000sapients/Trisduction, commit 4d37aa6, from which every parent file was read.

Appendix B. The seal, verbatim · La_Rayba_Fih_v2_0_0_Seal.lean, sha256 d1c1eb65d77844abd74338f4aa189920f2acf45b0c6b07fa17b6e392ee0a5236

The whole seal, Parts 0 to IX and its pinned foot, as compiled and cited. The line numbers of the codebook are the file's. Extract the block between its fence markers; the sha256 of the extracted file is the one in the heading, and `lean` on it under core Lean 4.19.0 exits 0 with no message, every guard matched.

```
/-!
  LĀ RAYBA FĪH — THE SEAL · v2.0.0
  The kernel of the paper "Lā Rayba Fīh: The Seven Millennium Closures Read as Balance, Witness and
  Preserved Record — the Form by Theorem, the Value by the Act, the Dominion with the Ground".
  Core Lean 4.19.0, standalone: no import, no sorry, no axiom declared, no native_decide. Every cone is pinned
  by #guard_msgs at the foot, so any drift in any cone fails the build.

  WHAT THE SEAL PROVES. The paper's four sentences, over every row of the normal form (a record that carries
  the value, two admissible configurations sharing the record and splitting on the value), and the paper's
  pair, seven and precedent on constructed rows:
    I    THE ACT IS ESSENTIAL AND IRREDUCIBLE. The value of a frame is exactly the act at it; nothing every
         configuration shares and no reading of the record supplies the act; the open framing, "derive the
         value from what every configuration shares", is refuted. This is lā rayba fīh in the mathematical
         register: the doubt is the open framing, and it is a theorem's negation.
    II   TWO KINDS AND NO THIRD. The formal kind and the physical kind are each exactly the value, the formal
         is never alone, and one refuter refutes both.
    III  A SIMPLE WITNESS PASSES EVERY WALL. A witness, a premise that forces the value and is present where
         the value holds, is not shared, reads past the record, and delivers the act; a wall on every
         witness is the negation.
    IV   THE FORMAL REALM ALONE HAS NO FINAL SAY. A ladder of sentences and models proves the value exactly
         when it proves the bit, enters the value by the act's door, reads past the record when it proves
         the value, and is blocked when its models pin only the record.
    V    THE PAIR. The Riemann row and the P versus NP row, built as rows, each closed to the act.
    VI   THE SEVEN. For any seven rows: each closed to its bit, two kinds each, no shared premise forces any,
         all 128 patterns of the seven values realized, and no principle true of every configuration
         forces all seven values.
    VII  THE PRECEDENT. A functional strictly decreasing off the seat and scale-free gives the act; the
         self-similar flow rests and admits no witness; the round world carries the witness and its stuck
         twin of the same record does not.
    VIII THE REGISTER LAW. A hearing assigns verses to claims; every mathematical claim of the paper is
         proved for every hearing and from none: the scriptural register is a free parameter of the seal.
    IX   THE PAPER, in one theorem with nothing before its colon.
  WHAT THE SEAL DOES NOT PROVE: any Millennium value. The real rows are supplied by the parents; here the
  rows are the normal form's hypotheses and three constructed instances. ΔM = 0.
-/

namespace LaRaybaFih

universe u v w

/-! ## Part 0 · A row of the normal form -/

structure Row where
  X         : Type u
  Adm       : X → Prop
  Value     : X → Prop
  Same      : X → X → Prop
  recObj    : X → X
  rec_has   : ∀ x, Adm x → Value (recObj x)
  compliant : X
  twin      : X
  adm_c     : Adm compliant
  adm_t     : Adm twin
```

```
  same_ct   : Same compliant twin
  value_c   : Value compliant
  value_t   : ¬ Value twin

def ReadsRecord (r : Row.{u}) (Q : r.X → Prop) : Prop := ∀ x y, r.Same x y → (Q x ↔ Q y)
def Bit (r : Row.{u}) (x : r.X) : Prop := r.Value (r.recObj x) → r.Value x
def Forces (r : Row.{u}) (Q : r.X → Prop) : Prop := ∀ x, r.Adm x → Q x → r.Value x

/-- The act at a configuration: its admissibility with its bit supplied. -/
structure ActAt (r : Row.{u}) (x : r.X) : Prop where
  adm    : r.Adm x
  supply : Bit r x

/-! ## Part I · The act is essential and irreducible -/

theorem the_bit_is_the_value (r : Row.{u}) (x : r.X) (hx : r.Adm x) : Bit r x ↔ r.Value x :=
  (fun b => b (r.rec_has x hx), fun v _ => v)

theorem the_act_is_essential (r : Row.{u}) (x : r.X) (hx : r.Adm x) : r.Value x ↔ ActAt r x :=
  (fun v => (hx, fun _ => v), fun a => a.supply (r.rec_has x a.adm))

theorem nothing_shared_forces (r : Row.{u}) (Q : r.X → Prop) (hQ : ∀ x, Q x) : ¬ Forces r Q :=
  fun h => r.value_t (h r.twin r.adm_t (hQ r.twin))

theorem the_record_decides_nothing (r : Row.{u}) (D : r.X → Prop) (hR : ReadsRecord r D) :
    ¬ ∀ x, r.Adm x → (D x ↔ r.Value x) :=
  fun h => r.value_t ((h r.twin r.adm_t).mp ((hR r.compliant r.twin r.same_ct).mp
    ((h r.compliant r.adm_c).mpr r.value_c)))

theorem the_act_is_irreducible (r : Row.{u}) :
    ¬ ActAt r r.twin ∧
    (∀ Q : r.X → Prop, (∀ x, Q x) → ¬ ∀ x, r.Adm x → Q x → ActAt r x) ∧
    (∀ D : r.X → Prop, ReadsRecord r D → ¬ ∀ x, r.Adm x → (D x ↔ ActAt r x)) :=
  have nt : ¬ ActAt r r.twin := fun a => r.value_t ((the_act_is_essential r r.twin r.adm_t).mpr a)
  (nt, fun _ hQ h => nt (h r.twin r.adm_t (hQ r.twin)),
   fun _ hR h => nt ((h r.twin r.adm_t).mp ((hR r.compliant r.twin r.same_ct).mp
     ((h r.compliant r.adm_c).mpr (r.adm_c, fun _ => r.value_c)))))

/-- The open framing: derive the value from a premise every configuration shares. -/
def OpenFraming (r : Row.{u}) : Prop := ∃ Q : r.X → Prop, (∀ x, Q x) ∧ Forces r Q

theorem the_formal_cannot_complete_itself (r : Row.{u}) : ¬ OpenFraming r :=
  fun (Q, hQ, hF) => nothing_shared_forces r Q hQ hF

/-- Lā rayba fīh, in the mathematical register: the doubt is the open framing and the record's decision,
    and both are refuted. -/
theorem la_rayba_fih (r : Row.{u}) :
    ¬ OpenFraming r ∧ ¬ ∃ D : r.X → Prop, ReadsRecord r D ∧ ∀ x, r.Adm x → (D x ↔ r.Value x) :=
  (the_formal_cannot_complete_itself r, fun (D, hR, h) =>
   the_record_decides_nothing r D hR h)

/-! ## Part II · Two kinds and no third -/

def Formal (r : Row.{u}) (x : r.X) : Prop := Bit r x

structure Physical (r : Row.{u}) (x : r.X) : Prop where
  carries : r.Value x

theorem two_kinds_and_no_third (r : Row.{u}) (x : r.X) (hx : r.Adm x) :
    (Formal r x ↔ r.Value x) ∧ (Physical r x ↔ r.Value x) ∧
    (∀ Q : Prop, (Q → r.Value x) → Q → Formal r x ∧ Physical r x) ∧
    (¬ r.Value x → ¬ Formal r x ∧ ¬ Physical r x) :=
  ((fun f => f (r.rec_has x hx), fun v _ => v),
   (fun p => p.carries, fun v => ⟨v⟩),
   fun _ h q => (fun _ => h q, (h q)),
   fun hn => (fun f => hn (f (r.rec_has x hx)), fun p => hn p.carries))

/-- The formal kind is never alone: every formal delivery is a physical one, and conversely. -/
theorem the_formal_is_never_alone (r : Row.{u}) (x : r.X) (hx : r.Adm x) :
    Formal r x ↔ Physical r x :=
  (fun f => (f (r.rec_has x hx)), fun p _ => p.carries)

theorem one_refuter_refutes_both (r : Row.{u}) (x : r.X) (hx : r.Adm x) :
    ¬ r.Value x → ¬ Formal r x ∧ ¬ Physical r x :=
  (two_kinds_and_no_third r x hx).2.2.2

/-! ## Part III · A simple witness passes every wall -/

/-- A witness: a premise that forces the value and is present where the value holds. -/
structure Witness (r : Row.{u}) where
  W       : r.X → Prop
  forces  : Forces r W
  present : W r.compliant

theorem the_witness_is_not_shared (r : Row.{u}) (w : Witness r) : ¬ ∀ x, w.W x :=
  fun h => nothing_shared_forces r w.W h w.forces

theorem the_witness_reads_past_the_record (r : Row.{u}) (w : Witness r) :
    ¬ ReadsRecord r w.W :=
  fun hR => r.value_t (w.forces r.twin r.adm_t ((hR r.compliant r.twin r.same_ct).mp w.present))

theorem the_witness_delivers_the_act (r : Row.{u}) (w : Witness r) :
    ∀ x, r.Adm x → w.W x → ActAt r x :=
  fun x hx q => (hx, fun _ => w.forces x hx q)

theorem the_witness_passes_every_wall (r : Row.{u}) (w : Witness r) :
    (¬ ∀ x, w.W x) ∧ ¬ ReadsRecord r w.W ∧ ∀ x, r.Adm x → w.W x → ActAt r x :=
  (the_witness_is_not_shared r w, the_witness_reads_past_the_record r w,
   the_witness_delivers_the_act r w)

theorem a_wall_on_every_witness_is_the_negation (B : Prop) : (∀ Q : Prop, (Q → B) → ¬ Q) ↔ ¬ B :=
  (fun h => h B id, fun hn _ hq q => hn (hq q))

/-- The value is its own witness: the simplest witness there is. -/
def the_value_is_its_own_witness (r : Row.{u}) : Witness r :=
  (r.Value, fun _ _ v => v, r.value_c)

/-! ## Part IV · The formal realm alone has no final say -/

structure Ladder where
  Sent : Type v
  Prov : Sent → Prop
  imp  : Sent → Sent → Sent
  mp   : ∀ a b, Prov (imp a b) → Prov a → Prov b

structure NormalForm (T : Ladder.{v}) where
  V  : T.Sent
  B  : T.Sent
  vb : T.Prov (T.imp V B)
  bv : T.Prov (T.imp B V)

structure OnRow (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T) where
  M     : Type w
  Holds : M → T.Sent → Prop
  sound : ∀ s m, T.Prov s → Holds m s
  real  : M → r.X
  adm   : ∀ m, r.Adm (real m)
  reads : ∀ m, Holds m N.V ↔ r.Value (real m)

theorem no_escape (T : Ladder.{v}) (N : NormalForm T) : T.Prov N.V ↔ T.Prov N.B :=
  (T.mp _ _ N.vb, T.mp _ _ N.bv)

theorem one_door (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T) (R : OnRow.{u, v, w} r T N)
    (p : T.Prov N.V) (m : R.M) : ActAt r (R.real m) :=
  (R.adm m, fun _ => (R.reads m).mp (R.sound N.V m p))

theorem blocked_of_defiant_model (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{u, v, w} r T N) (m : R.M) (hd : ¬ r.Value (R.real m)) :
    ¬ T.Prov N.V ∧ ¬ T.Prov N.B :=
  have hv : ¬ T.Prov N.V := fun p => hd ((R.reads m).mp (R.sound N.V m p))
  (hv, fun pb => hv ((no_escape T N).mpr pb))

theorem record_bound_ladder_is_blocked (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{u, v, w} r T N)
    (bound : ∀ m y, r.Adm y → r.Same (R.real m) y → ∃ m', R.real m' = y)
    (m₀ : R.M) (h₀ : R.real m₀ = r.compliant) : ¬ T.Prov N.V ∧ ¬ T.Prov N.B :=
  have hs : r.Same (R.real m₀) r.twin := h₀ ▸ r.same_ct
  match bound m₀ r.twin r.adm_t hs with
  | ⟨m', h'⟩ => blocked_of_defiant_model r T N R m' (h' ▸ r.value_t)

theorem proving_ladder_reads_past_the_record (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{u, v, w} r T N) (p : T.Prov N.V) (m₀ : R.M) (h₀ : R.real m₀ = r.compliant) :
    ¬ ∀ m y, r.Adm y → r.Same (R.real m) y → ∃ m', R.real m' = y :=
  fun bound => (record_bound_ladder_is_blocked r T N R bound m₀ h₀).1 p

theorem the_formal_alone_has_no_final_say (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{u, v, w} r T N) :
    (T.Prov N.V ↔ T.Prov N.B) ∧
    (∀ m, T.Prov N.V → ActAt r (R.real m)) ∧
    (∀ m, T.Prov N.V → Formal r (R.real m) ∧ Physical r (R.real m)) ∧
    ((∀ m y, r.Adm y → r.Same (R.real m) y → ∃ m', R.real m' = y) →
      ∀ m₀, R.real m₀ = r.compliant → ¬ T.Prov N.V ∧ ¬ T.Prov N.B) :=
  (no_escape T N, fun m p => one_door r T N R p m,
   fun m p => (two_kinds_and_no_third r (R.real m) (R.adm m)).2.2.1 (T.Prov N.V)
     (fun q => (R.reads m).mp (R.sound N.V m q)) p,
   fun bound m₀ h₀ => record_bound_ladder_is_blocked r T N R bound m₀ h₀)

/-! ## Part V · The pair: the master row and the humble servant, as rows -/
/-! ### V.1 Riemann -/
namespace RH
structure Pt where
  x : Int
  t : Nat
deriving DecidableEq
def onLine (z : Pt) : Prop := z.x = 1
```

```
def fold (z : Pt) : Pt := (2 - z.x, z.t)
def reg (z : Pt) : Pt := (1, z.t)
abbrev World := Pt → Prop
def FoldClosed (Z : World) : Prop := ∀ z, Z z → Z (fold z)
def RHolds (Z : World) : Prop := ∀ z, Z z → onLine z
def recordOf (Z : World) : World := fun r => ∃ s, Z s ∧ reg s = r
def SameRecord (Z Z' : World) : Prop := ∀ r, recordOf Z r ↔ recordOf Z' r
theorem fold_fixed_iff (z : Pt) : fold z = z ↔ onLine z := by cases z
with | mk x t => show Pt.mk (2 - x) t = Pt.mk x t ↔ x = 1; constructor;
intro h; have hx := congrArg Pt.x h; change 2 - x = x at hx; omega; intro
h; subst h; rfl
theorem record_on_line (Z : World) : RHolds (recordOf Z) := fun _ (s, _,
hs) => by subst hs; rfl
abbrev W1 : World := fun z => z = (1, 14)
abbrev W2 : World := fun z => z = (0, 14) ∨ z = (2, 14)
theorem W1_closed : FoldClosed W1 := by intro z hz; have h : z = (1, 14)
:= hz; subst h; show fold (1, 14) = (1, 14); decide
theorem W1_rh : RHolds W1 := by intro z hz; have h : z = (1, 14) := hz;
subst h; rfl
theorem W2_closed : FoldClosed W2 := by intro z hz; rcases hz with h | h;
subst h; exact Or.inr rfl; subst h; exact Or.inl rfl
theorem W2_not_rh : ¬ RHolds W2 := fun h => absurd (h (0, 14) (Or.inl
rfl)) (by decide : ¬ ((0 : Int) = 1))
theorem same_record : SameRecord W1 W2 := by intro r; constructor; intro
(s, hs, hr); have h : s = (1, 14) := hs; subst h; exact ((0, 14), Or.inl
rfl, hr); intro (s, hs, hr); rcases hs with h | h <;> subst h <;> exact
((1, 14), rfl, hr)
def row : Row.{0} where
  X := World
  Adm := FoldClosed
  Value := RHolds
  Same := SameRecord
  recObj := recordOf
  rec_has := fun Z _ => record_on_line Z
  compliant := W1
  twin := W2
  adm_c := W1_closed
  adm_t := W2_closed
  same_ct := same_record
  value_c := W1_rh
  value_t := W2_not_rh
end RH

/-! ### V.2 P vs NP -/
namespace PNP
structure Machine where
  Alg : Type
  Inst : Type
  size : Inst → Nat
  out : Alg → Inst → Bool
  time : Alg → Inst → Nat
  sat : Inst → Bool
def bound (c k n : Nat) : Nat := c * n ^ k + c
def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
def NothingEscapes (M : Machine) : Prop := ∀ (A : M.Alg) (c k : Nat), ∃
x, ¬ Clean M A c k x
def Rec (M : Machine) : Machine := { M with out := fun _ x => !(M.sat x) }
theorem bnot_ne (b : Bool) : (!b) ≠ b := by cases b <;> decide
theorem record_closes (M : Machine) (x₀ : M.Inst) : NothingEscapes (Rec
M) := fun _ _ _ => (x₀, fun hc => bnot_ne (M.sat x₀) hc.1)
theorem complement_free : ¬ ∃ b : Bool, (!b) = b := fun (b, h) => bnot_ne
b h
def closedWorld : Machine := (Nat, Unit, fun _ => 0, fun _ _ => false,
fun _ _ => 0, fun _ => true)
def stageWorld : Machine := (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A
0, fun _ _ => 0, fun _ => true)
theorem closed_world_closes : NothingEscapes closedWorld := fun _ _ _ =>
((), fun (h, _) => Bool.noConfusion h)
theorem stage_escapes : ¬ NothingEscapes stageWorld := fun h => match h
(0 : Nat) 0 0 with | (_, hx) => hx (rfl, Nat.zero_le _)
theorem same_rec : Rec closedWorld = Rec stageWorld := rfl
def row : Row.{1} where
  X := Machine
  Adm := fun M => Nonempty M.Inst
  Value := NothingEscapes
  Same := fun M M' => Rec M = Rec M'
  recObj := Rec
  rec_has := fun M h => match h with | (x₀) => record_closes M x₀
  compliant := closedWorld
  twin := stageWorld
  adm_c := ((),)
  adm_t := ((),)
  same_ct := same_rec
  value_c := closed_world_closes
  value_t := stage_escapes
end PNP

/-! ### V.3 Poincaré -/
namespace PC
def steps {α : Type} (f : α → α) : Nat → α → α | 0, x => x | k + 1, x =>
steps f k (f x)
structure Flow where
  State : Type
  step : State → State
  round : State
  rescale : State → State
  deq : ∀ s, Decidable (s = round)
def ComesToRest (F : Flow) (s : F.State) : Prop := ∃ k, steps F.step k s =
F.round
def Act (F : Flow) : Prop := ∀ s, ComesToRest F s
structure Witness (F : Flow) where
  W : F.State → Nat
  decrease : ∀ s, s ≠ F.round → W (F.step s) < W s
  scaleFree : ∀ s, W (F.rescale s) = W s
theorem descent_bound (F : Flow) (P : Witness F) : ∀ n s, P.W s ≤ n →
ComesToRest F s
  | 0, s, hs => (match F.deq s with
      | isTrue e => (0, e)
      | isFalse ne => absurd (Nat.lt_of_lt_of_le (P.decrease s ne) hs)
      (Nat.not_lt_zero _))
  | n + 1, s, hs => (match F.deq s with
      | isTrue e => (0, e)
      | isFalse ne => (match descent_bound F P n (F.step s)
      (Nat.le_of_lt_succ (Nat.lt_of_lt_of_le (P.decrease s ne) hs)) with
          | (k, hk) => (k + 1, hk)))
theorem witness_gives_the_act (F : Flow) (P : Witness F) : Act F := fun s
=> descent_bound F P (P.W s) s (Nat.le_refl _)
def soliton : Flow where
  State := Bool
  step := fun _ => true
  round := true
  rescale := fun _ => true
  deq := fun s => decEq s true
theorem soliton_rests : Act soliton := fun _ => (1, rfl)
theorem no_witness_on_the_soliton : ¬ Nonempty (Witness soliton) := fun
(P) => have d : P.W true < P.W false := P.decrease false (fun h =>
Bool.noConfusion h); have e : P.W true = P.W false := P.scaleFree false;
Nat.lt_irrefl _ (e ▸ d)
def recFlow (F : Flow) : Flow := { F with step := fun _ => F.round }
theorem rec_rests (F : Flow) : Act (recFlow F) := fun _ => (1, rfl)
def roundWorld : Flow where
  State := Bool
  step := fun _ => true
  round := true
  rescale := fun s => s
  deq := fun s => decEq s true
def stuckWorld : Flow where
  State := Bool
  step := fun s => s
  round := true
  rescale := fun s => s
  deq := fun s => decEq s true
def roundWitness : Witness roundWorld where
  W := fun s => cond s 0 1
  decrease := fun s hs => match s, hs with | true, h => absurd rfl h |
  false, _ => Nat.zero_lt_one
  scaleFree := fun _ => rfl
theorem steps_stuck : ∀ k : Nat, steps stuckWorld.step k false = false | 0
=> rfl | k + 1 => steps_stuck k
theorem stuck_does_not_rest : ¬ Act stuckWorld := fun h => match h false
with | (k, hk) => Bool.noConfusion ((steps_stuck k).symm.trans hk)
theorem same_rec : recFlow roundWorld = recFlow stuckWorld := rfl
def row : Row.{1} where
  X := Flow
  Adm := fun _ => True
  Value := Act
  Same := fun F F' => recFlow F = recFlow F'
  recObj := recFlow
  rec_has := fun F _ => rec_rests F
  compliant := roundWorld
  twin := stuckWorld
  adm_c := trivial
  adm_t := trivial
  same_ct := same_rec
  value_c := witness_gives_the_act roundWorld roundWitness
  value_t := stuck_does_not_rest
end PC

theorem the_pair :
    (∀ x, RH.row.Adm x → (RH.row.Value x ↔ ActAt RH.row x)) ∧
    (∀ x, PNP.row.Adm x → (PNP.row.Value x ↔ ActAt PNP.row x)) ∧
    ¬ OpenFraming RH.row ∧ ¬ OpenFraming PNP.row ∧
    (ActAt RH.row RH.W1 ∧ ¬ ActAt RH.row RH.W2 ∧ RH.SameRecord RH.W1
    RH.W2) ∧
    (ActAt PNP.row PNP.closedWorld ∧ ¬ ActAt PNP.row PNP.stageWorld ∧
      PNP.Rec PNP.closedWorld = PNP.Rec PNP.stageWorld) :=
  (fun x hx => the_act_is_essential RH.row x hx, fun x hx =>
  the_act_is_essential PNP.row x hx,
   the_formal_cannot_complete_itself RH.row,
   the_formal_cannot_complete_itself PNP.row,
   ((RH.W1_closed, fun _ => RH.W1_rh), (the_act_is_irreducible RH.row).1,
   RH.same_record),
   (((((), fun _ => PNP.closed_world_closes), (the_act_is_irreducible
   PNP.row).1, PNP.same_rec))

/-! ## Part VI · The seven rows -/

theorem seven_closed_to_one_bit (rows : Fin 7 → Row.{u}) :
    ∀ i x, (rows i).Adm x → (Bit (rows i) x ↔ (rows i).Value x) :=
  fun i x hx => the_bit_is_the_value (rows i) x hx

theorem seven_two_kinds (rows : Fin 7 → Row.{u}) :
    ∀ i x, (rows i).Adm x →
      (Formal (rows i) x ↔ (rows i).Value x) ∧ (Physical (rows i) x ↔
      (rows i).Value x) ∧
      (¬ (rows i).Value x → ¬ Formal (rows i) x ∧ ¬ Physical (rows i) x)
      :=
  fun i x hx => ((two_kinds_and_no_third (rows i) x hx).1,
  (two_kinds_and_no_third (rows i) x hx).2.1,
    (two_kinds_and_no_third (rows i) x hx).2.2.2)

theorem seven_keys (rows : Fin 7 → Row.{u}) : ∀ i, ¬ OpenFraming (rows i)
:=
  fun i => the_formal_cannot_complete_itself (rows i)
```

```
theorem a_coordinate_takes_either_value (r : Row.{u}) :
    ∀ b : Bool, r.Adm (cond b r.compliant r.twin) ∧ (r.Value (cond b
      r.compliant r.twin) ↔ b = true)
  | true  => (r.adm_c, (fun _ => rfl, fun _ => r.value_c))
  | false => (r.adm_t, (fun v => absurd v r.value_t, fun h =>
    Bool.noConfusion h))

/-- All 128 patterns of the seven values are realized on the product of
the rows' own frames. -/
theorem the_seven_values_are_independent (rows : Fin 7 → Row.{u}) :
    ∀ p : Fin 7 → Bool, ∃ c : (i : Fin 7) → (rows i).X,
      ∀ i, (rows i).Adm (c i) ∧ ((rows i).Value (c i) ↔ p i = true) :=
  fun p => (fun i => cond (p i) (rows i).compliant (rows i).twin,
            fun i => a_coordinate_takes_either_value (rows i) (p i))

/-- No principle true of every configuration of the seven forces all
seven values. -/
theorem no_unified_principle (rows : Fin 7 → Row.{u}) :
    ¬ ∃ P : ((i : Fin 7) → (rows i).X) → Prop,
      (∀ c, P c) ∧ ∀ c, P c → ∀ i, (rows i).Adm (c i) → (rows i).Value (c
      i) :=
  fun (_, hP, hF) =>
    have i₀ : Fin 7 := (0, Nat.zero_lt_succ 6)
    (rows i₀).value_t (hF (fun i => (rows i).twin) (hP _) i₀ (rows
    i₀).adm_t)

theorem the_seven (rows : Fin 7 → Row.{u}) :
    (∀ i x, (rows i).Adm x → (Bit (rows i) x ↔ (rows i).Value x)) ∧
    (∀ i x, (rows i).Adm x → (Formal (rows i) x ↔ (rows i).Value x) ∧
      (Physical (rows i) x ↔ (rows i).Value x) ∧
      (¬ (rows i).Value x → ¬ Formal (rows i) x ∧ ¬ Physical (rows i) x))
      ∧
    (∀ i, ¬ OpenFraming (rows i)) ∧
    (∀ p : Fin 7 → Bool, ∃ c : (i : Fin 7) → (rows i).X,
      ∀ i, (rows i).Adm (c i) ∧ ((rows i).Value (c i) ↔ p i = true)) ∧
    ¬ ∃ P : ((i : Fin 7) → (rows i).X) → Prop,
      (∀ c, P c) ∧ ∀ c, P c → ∀ i, (rows i).Adm (c i) → (rows i).Value (c
      i) :=
  (seven_closed_to_one_bit rows, seven_two_kinds rows, seven_keys rows,
   the_seven_values_are_independent rows, no_unified_principle rows)

/-! ## Part VII · The precedent: the crossed row -/

theorem the_precedent :
    (∀ F : PC.Flow, Nonempty (PC.Witness F) → ActAt PC.row F) ∧
    (PC.Act PC.soliton ∧ ¬ Nonempty (PC.Witness PC.soliton)) ∧
    (ActAt PC.row PC.roundWorld ∧ ¬ ActAt PC.row PC.stuckWorld ∧
      PC.recFlow PC.roundWorld = PC.recFlow PC.stuckWorld) ∧
    (¬ ∀ F : PC.Flow, Nonempty (PC.Witness F)) :=
  (fun F (P) => (trivial, fun _ => PC.witness_gives_the_act F P),
   (PC.soliton_rests, PC.no_witness_on_the_soliton),
   ((trivial, fun _ => PC.witness_gives_the_act PC.roundWorld
    PC.roundWitness),
    (the_act_is_irreducible PC.row).1, PC.same_rec),
   fun h => PC.no_witness_on_the_soliton (h PC.soliton))

/-! ## Part VIII · The register law -/

inductive Register where
  | mathematical
  | scriptural

/-- A hearing assigns a verse to each claim. It is data, never a
hypothesis. -/
structure Hearing (Claim : Type) where
  verse : Claim → String

/-- Heard afterward: a claim proved in the mathematical register holds
for every hearing and from none. -/
theorem heard_afterward {Claim : Type} (M : Prop) (hM : M) : ∀ _ :
Hearing Claim, M :=
  fun _ => hM

/-- The register law: no hearing is a premise of any claim; the two
registers do not cross. -/
theorem the_registers_do_not_cross {Claim : Type} (M : Prop) :
    (∃ _ : Hearing Claim, M) ↔ M :=
  (fun (_, m) => m, fun m => (⟨fun _ => ""⟩, m))

/-! ## Part IX · The paper, in one theorem -/

theorem the_paper :
    (∀ r : Row.{u}, (∀ x, r.Adm x → (r.Value x ↔ ActAt r x)) ∧
      (¬ ActAt r r.twin ∧
       (∀ Q : r.X → Prop, (∀ x, Q x) → ¬ ∀ x, r.Adm x → Q x → ActAt r x) ∧
       (∀ D : r.X → Prop, ReadsRecord r D → ¬ ∀ x, r.Adm x → (D x ↔ ActAt
        r x))) ∧
      ¬ OpenFraming r ∧
      ¬ ∃ D : r.X → Prop, ReadsRecord r D ∧ ∀ x, r.Adm x → (D x ↔ r.Value
      x)) ∧
    (∀ (r : Row.{u}) (x : r.X), r.Adm x →
      (Formal r x ↔ r.Value x) ∧ (Physical r x ↔ r.Value x) ∧
      (∀ Q : Prop, (Q → r.Value x) → Q → Formal r x ∧ Physical r x) ∧
      (¬ r.Value x → ¬ Formal r x ∧ ¬ Physical r x)) ∧
    (∀ (r : Row.{u}) (w : Witness r),
      (¬ ∀ x, w.W x) ∧ ¬ ReadsRecord r w.W ∧ ∀ x, r.Adm x → w.W x → ActAt
      r x) ∧
    (∀ B : Prop, (∀ Q : Prop, (Q → B) → ¬ Q) ↔ ¬ B) ∧
    (∀ (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T) (R : OnRow.{u,
    v, w} r T N),
      (T.Prov N.V ↔ T.Prov N.B) ∧
      (∀ m, T.Prov N.V → ActAt r (R.real m)) ∧
      (∀ m, T.Prov N.V → Formal r (R.real m) ∧ Physical r (R.real m)) ∧
      ((∀ m y, r.Adm y → r.Same (R.real m) y → ∃ m', R.real m' = y) →
        ∀ m₀, R.real m₀ = r.compliant → ¬ T.Prov N.V ∧ ¬ T.Prov N.B)) ∧
    (∀ rows : Fin 7 → Row.{u},
      (∀ i, ¬ OpenFraming (rows i)) ∧
      (∀ p : Fin 7 → Bool, ∃ c : (i : Fin 7) → (rows i).X,
        ∀ i, (rows i).Adm (c i) ∧ ((rows i).Value (c i) ↔ p i = true)) ∧
      ¬ ∃ P : ((i : Fin 7) → (rows i).X) → Prop,
        (∀ c, P c) ∧ ∀ c, P c → ∀ i, (rows i).Adm (c i) → (rows i).Value
        (c i)) ∧
    (∀ (Claim : Type) (M : Prop), (∃ _ : Hearing Claim, M) ↔ M) :=
  (fun r => (fun x hx => the_act_is_essential r x hx,
  the_act_is_irreducible r,
             the_formal_cannot_complete_itself r, (la_rayba_fih r).2),
   fun r x hx => two_kinds_and_no_third r x hx,
   fun r w => the_witness_passes_every_wall r w,
   a_wall_on_every_witness_is_the_negation,
   fun r T N R => the_formal_alone_has_no_final_say r T N R,
   fun rows => (seven_keys rows, the_seven_values_are_independent rows,
   no_unified_principle rows),
   fun Claim M => the_registers_do_not_cross (Claim := Claim) M)

/-- The paper on its constructed rows: the pair and the precedent, with
nothing before the colon. -/
theorem the_paper_on_the_rows :
    ((∀ x, RH.row.Adm x → (RH.row.Value x ↔ ActAt RH.row x)) ∧
     (∀ x, PNP.row.Adm x → (PNP.row.Value x ↔ ActAt PNP.row x)) ∧
     ¬ OpenFraming RH.row ∧ ¬ OpenFraming PNP.row ∧
     (ActAt RH.row RH.W1 ∧ ¬ ActAt RH.row RH.W2 ∧ RH.SameRecord RH.W1
     RH.W2) ∧
     (ActAt PNP.row PNP.closedWorld ∧ ¬ ActAt PNP.row PNP.stageWorld ∧
       PNP.Rec PNP.closedWorld = PNP.Rec PNP.stageWorld)) ∧
    ((∀ F : PC.Flow, Nonempty (PC.Witness F) → ActAt PC.row F) ∧
     (PC.Act PC.soliton ∧ ¬ Nonempty (PC.Witness PC.soliton)) ∧
     (ActAt PC.row PC.roundWorld ∧ ¬ ActAt PC.row PC.stuckWorld ∧
       PC.recFlow PC.roundWorld = PC.recFlow PC.stuckWorld) ∧
     (¬ ∀ F : PC.Flow, Nonempty (PC.Witness F))) :=
  (the_pair, the_precedent)

end LaRaybaFih

/-! Cones, pinned -/

/-- info: 'LaRaybaFih.ReadsRecord' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.ReadsRecord
/-- info: 'LaRaybaFih.Bit' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.Bit
/-- info: 'LaRaybaFih.Forces' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.Forces
/-- info: 'LaRaybaFih.the_bit_is_the_value' does not depend on any axioms
-/
#guard_msgs in #print axioms LaRaybaFih.the_bit_is_the_value
/-- info: 'LaRaybaFih.the_act_is_essential' does not depend on any axioms
-/
#guard_msgs in #print axioms LaRaybaFih.the_act_is_essential
/-- info: 'LaRaybaFih.nothing_shared_forces' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.nothing_shared_forces
/-- info: 'LaRaybaFih.the_record_decides_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_record_decides_nothing
/-- info: 'LaRaybaFih.the_act_is_irreducible' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_act_is_irreducible
/-- info: 'LaRaybaFih.OpenFraming' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.OpenFraming
/-- info: 'LaRaybaFih.the_formal_cannot_complete_itself' does not depend
on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_formal_cannot_complete_itself
/-- info: 'LaRaybaFih.la_rayba_fih' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.la_rayba_fih
/-- info: 'LaRaybaFih.Formal' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.Formal
/-- info: 'LaRaybaFih.two_kinds_and_no_third' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.two_kinds_and_no_third
/-- info: 'LaRaybaFih.the_formal_is_never_alone' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_formal_is_never_alone
/-- info: 'LaRaybaFih.one_refuter_refutes_both' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.one_refuter_refutes_both
/-- info: 'LaRaybaFih.the_witness_is_not_shared' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_witness_is_not_shared
/-- info: 'LaRaybaFih.the_witness_reads_past_the_record' does not depend
on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_witness_reads_past_the_record
/-- info: 'LaRaybaFih.the_witness_delivers_the_act' does not depend on
any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_witness_delivers_the_act
/-- info: 'LaRaybaFih.the_witness_passes_every_wall' does not depend on
any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_witness_passes_every_wall
/-- info: 'LaRaybaFih.a_wall_on_every_witness_is_the_negation' does not
depend on any axioms -/
#guard_msgs in #print axioms
LaRaybaFih.a_wall_on_every_witness_is_the_negation
/-- info: 'LaRaybaFih.the_value_is_its_own_witness' does not depend on
any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_value_is_its_own_witness
/-- info: 'LaRaybaFih.no_escape' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.no_escape
/-- info: 'LaRaybaFih.one_door' does not depend on any axioms -/
```

```
#guard_msgs in #print axioms LaRaybaFih.one_door
/-- info: 'LaRaybaFih.blocked_of_defiant_model' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.blocked_of_defiant_model
/-- info: 'LaRaybaFih.record_bound_ladder_is_blocked' does not depend on
any axioms -/
#guard_msgs in #print axioms LaRaybaFih.record_bound_ladder_is_blocked
/-- info: 'LaRaybaFih.proving_ladder_reads_past_the_record' does not
depend on any axioms -/
#guard_msgs in #print axioms
LaRaybaFih.proving_ladder_reads_past_the_record
/-- info: 'LaRaybaFih.the_formal_alone_has_no_final_say' does not depend
on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_formal_alone_has_no_final_say
/-- info: 'LaRaybaFih.RH.onLine' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.onLine
/-- info: 'LaRaybaFih.RH.fold' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.fold
/-- info: 'LaRaybaFih.RH.reg' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.reg
/-- info: 'LaRaybaFih.RH.FoldClosed' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.FoldClosed
/-- info: 'LaRaybaFih.RH.RHolds' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.RHolds
/-- info: 'LaRaybaFih.RH.recordOf' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.recordOf
/-- info: 'LaRaybaFih.RH.SameRecord' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.SameRecord
/-- info: 'LaRaybaFih.RH.fold_fixed_iff' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms LaRaybaFih.RH.fold_fixed_iff
/-- info: 'LaRaybaFih.RH.record_on_line' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.record_on_line
/-- info: 'LaRaybaFih.RH.W1_closed' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.W1_closed
/-- info: 'LaRaybaFih.RH.W1_rh' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.W1_rh
/-- info: 'LaRaybaFih.RH.W2_closed' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.W2_closed
/-- info: 'LaRaybaFih.RH.W2_not_rh' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.W2_not_rh
/-- info: 'LaRaybaFih.RH.same_record' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.same_record
/-- info: 'LaRaybaFih.RH.row' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.RH.row
/-- info: 'LaRaybaFih.PNP.bound' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.bound
/-- info: 'LaRaybaFih.PNP.Clean' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.Clean
/-- info: 'LaRaybaFih.PNP.NothingEscapes' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.NothingEscapes
/-- info: 'LaRaybaFih.PNP.Rec' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.Rec
/-- info: 'LaRaybaFih.PNP.bnot_ne' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.bnot_ne
/-- info: 'LaRaybaFih.PNP.record_closes' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.record_closes
/-- info: 'LaRaybaFih.PNP.complement_free' does not depend on any axioms
-/
#guard_msgs in #print axioms LaRaybaFih.PNP.complement_free
/-- info: 'LaRaybaFih.PNP.closedWorld' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.closedWorld
/-- info: 'LaRaybaFih.PNP.stageWorld' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.stageWorld
/-- info: 'LaRaybaFih.PNP.closed_world_closes' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.closed_world_closes
/-- info: 'LaRaybaFih.PNP.stage_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.stage_escapes
/-- info: 'LaRaybaFih.PNP.same_rec' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.same_rec
/-- info: 'LaRaybaFih.PNP.row' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PNP.row
/-- info: 'LaRaybaFih.PC.steps' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.steps
/-- info: 'LaRaybaFih.PC.ComesToRest' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.ComesToRest
/-- info: 'LaRaybaFih.PC.Act' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.Act
/-- info: 'LaRaybaFih.PC.descent_bound' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.descent_bound
/-- info: 'LaRaybaFih.PC.witness_gives_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.witness_gives_the_act
/-- info: 'LaRaybaFih.PC.soliton' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.soliton
/-- info: 'LaRaybaFih.PC.soliton_rests' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.soliton_rests
/-- info: 'LaRaybaFih.PC.no_witness_on_the_soliton' does not depend on
any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.no_witness_on_the_soliton
/-- info: 'LaRaybaFih.PC.recFlow' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.recFlow
/-- info: 'LaRaybaFih.PC.rec_rests' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.rec_rests
/-- info: 'LaRaybaFih.PC.roundWorld' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.roundWorld
/-- info: 'LaRaybaFih.PC.stuckWorld' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.stuckWorld
/-- info: 'LaRaybaFih.PC.roundWitness' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.roundWitness
/-- info: 'LaRaybaFih.PC.steps_stuck' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.steps_stuck
/-- info: 'LaRaybaFih.PC.stuck_does_not_rest' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.stuck_does_not_rest
/-- info: 'LaRaybaFih.PC.same_rec' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.same_rec
/-- info: 'LaRaybaFih.PC.row' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.PC.row
/-- info: 'LaRaybaFih.the_pair' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_pair
/-- info: 'LaRaybaFih.seven_closed_to_one_bit' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.seven_closed_to_one_bit
/-- info: 'LaRaybaFih.seven_two_kinds' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.seven_two_kinds
/-- info: 'LaRaybaFih.seven_keys' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.seven_keys
/-- info: 'LaRaybaFih.a_coordinate_takes_either_value' does not depend on
any axioms -/
#guard_msgs in #print axioms LaRaybaFih.a_coordinate_takes_either_value
/-- info: 'LaRaybaFih.the_seven_values_are_independent' does not depend
on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_seven_values_are_independent
/-- info: 'LaRaybaFih.no_unified_principle' does not depend on any axioms
-/
#guard_msgs in #print axioms LaRaybaFih.no_unified_principle
/-- info: 'LaRaybaFih.the_seven' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_seven
/-- info: 'LaRaybaFih.the_precedent' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_precedent
/-- info: 'LaRaybaFih.heard_afterward' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.heard_afterward
/-- info: 'LaRaybaFih.the_registers_do_not_cross' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_registers_do_not_cross
/-- info: 'LaRaybaFih.the_paper' does not depend on any axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_paper
/-- info: 'LaRaybaFih.the_paper_on_the_rows' does not depend on any
axioms -/
#guard_msgs in #print axioms LaRaybaFih.the_paper_on_the_rows
```

Appendix C. The ZFC Block, final v2.0.0, reflowed edition, verbatim · ZFC_Block_Final_v2_0_0_Reflowed.lean, sha256 9dcf0cfe350073f69e45b9ff0cfc72096f027a0d9bf3a1667ae1cb5f43d896dc

The ZFC Block as compiled and cited in Sections 5 and 6, Parts 0 to XIV with its pinned foot; the three edits beyond reflow are listed in the Disclosure. Extract the block between its fence markers; the sha256 of the extracted file is the one in the heading, and `lean` on it under core Lean 4.19.0 exits 0, its one message the unused-variable warning noted in the Disclosure.

```
/-!
  THE ZFC BLOCK, FINAL v2.0.0 · HERMETICALLY SEALED · TWO BOXES TOGETHER —
  TERMINAL UNDERTAKER
  The Riemann row, the P versus NP row, and the Poincaré row with
  Perelman as Grand Witness, in one kernel.
  Core Lean 4.19.0, standalone: no import, no sorry, no axiom declared,
  no native_decide. Every theorem's axiom cone
  is pinned by #guard_msgs at the foot of the file, so any drift in any
  cone fails the build. Hostile builds are
  scripted beside the file (ZFC_Block_Hardened_v1_4_0_controls.sh); each
  plants one break and must be refused.

  WHAT IS PROVED, ON ALL THREE ROWS
    I    THE AIRTIGHT CHAIN before the posit. Exit I, shared resources:
    nothing true of every configuration forces the
         value. Exit II, the record: nothing that reads only the record
         forces or decides it. Exit III, the bypass:
         no premise delivers the value while the bit fails. NO FOURTH
         KIND: a premise that forces the value, holding
         where the value holds, is not shared, does not read only the
         record, and delivers the bit.
    II   TWO KINDS AND NO THIRD. The formal kind and the physical kind
    are each exactly the value; every witness of
         the value, from any premise, is a witness of both kinds; one
         refuter refutes both.
    III  ZFC READ AS A LADDER: sentences, provability, modus ponens, and
    models in which every theorem holds, each
         model realizing a configuration of the row, with the normal form
         proved inside the ladder.
         NO ESCAPE: the ladder proves the value exactly when it proves
         the bit, and every ladder proof of the value
         is of both kinds. THE BLOCK: a model realizing a defiant
         configuration blocks the ladder from the value and
         from the bit; with Gödel's completeness theorem carried as a
         hypothesis, exactly then. RECORD-BOUND: a ladder
         whose models pin only the record is blocked; a ladder that
         proves the value reads past the record.
    IV   DECIDED BY THE MODELS. On each row, beside the same airtight
    chain, a ladder whose models realize only the
         compliant configuration proves the value and the bit, and a
         ladder with a model realizing the defiant twin
         proves neither.
    V    PERELMAN, THE GRAND WITNESS, typed as the Poincaré Precedent
    types him: a functional on states, strictly
```

```
        decreasing along the step off the seat and invariant under the
        rescaling. The witness gives the act. It is
        strictly inside the act: the self-similar flow rests and admits
        no witness. The round world carries it and
        its stuck twin of one record does not. The witness route has
        exactly the shape the chain allows a route that
        succeeds: not shared, reading past the record, delivering the
        bit. On the one crossed row the crossing went
        through the bit, never around it.
  VI    BLESSING IS ADOPTION: adopting the bit as an axiom proves
  exactly what adopting the sentence proves.
  VII   THE FENCE: a wall on every true premise is the negation;
  irrefutability by a Σ₁-complete ladder is a proof.
  VIII THE SLOGAN (Part IX), "deny our proof as a valid proof, and ZFC
  stays blocked", at theorem grade with its scope in the
        same clause. DENY THE BIT at the actual configuration, and every
        sound ladder carrying the normal form, ZFC
        among them, stays blocked from the value and from the bit.
        REJECT OUR PROOF AS A KIND, every route that
        delivers the bit, and every ladder proof of the value is
        rejected with it. The implication runs from the
        denial to the block: a ladder can be blocked while the bit holds.
  IX    THE ROUTE LEDGER (Part X), typed by evidence, after the ledger
  of Zenodo 23308381 (§XIX.e). The semantic
        ladders are read on their rows as sound ladders carrying the
        normal form, so Part III's hypotheses are met by
        constructed terms, in the proving case and in the blocked case.
        THE LADDER ROUTE IS OPEN: on every row one
        such ladder proves the value and the bit, and another is blocked
        from both. NO BLOCK FROM THE ROW: the
        unrestricted block is refuted on every row, not merely unproved.
        THE CHAIN DOES NOT BLOCK: on all three rows
        the airtight chain and a ladder proof of the value stand
        together. Every printed status carries its theorem
        and the opposite status is refuted, so no status can be flipped:
        the ladder from the record blocked, the
        foundation as a system open, independence with the value false
        blocked.
  X     THE ACT, BY THEOREM (Part XI), after the record's `ActualZeros`:
  the act at a configuration is its
        admissibility with its bit supplied, as the fields of a type.
        FROM THE ACT, THE VALUE, of both kinds. EVERY
        WAY IS THE ACT: every proof of the value, from any premise
        whatever, is a supply of the act. THE ACT IS THE
        WEAKEST FORCING PREMISE: the posit spends one bit and no more.
        THE ACT IS IRREDUCIBLE: nothing every
        configuration shares and no reading of the record supplies it.
        ONE DOOR: every ladder that proves the value
        enters by the act's door, a foundation's included. THE
        PRECEDENT: Perelman's witness supplies the act on every
        flow that carries it, and is constructed on the round world.
        BLESSING THE ACT IS ADOPTING THE VALUE, at
        exactly the value's price.
  XI    THE UNCONDITIONAL VISIBLE BLOCK (Part XII): the actual SAT
  machine and the actual zeros as rows; the act at
        the actual configuration as the posit; deny the bit at the
        actual, and every sound ladder whose actual model
        is the actual configuration — ZFC's standard model among them if
        consistent — stays blocked; reject the kind
        Formal = Bit at the actual, and you reject every ZFC proof of
        the value with it; the block is visible as a
        closed term: Blocked ↔ ¬ Value(actual) and Blocked does not
        imply ¬ Bit(actual) fails, it implies ¬ Prov.

  WHAT IS NOT PROVED, AND DOES NOT FOLLOW FROM THESE PREMISES: that ZFC
  proves neither the Riemann value nor the
  P ≠ NP value. By III the ladder is blocked exactly when a model of ZFC
  realizes the defiant twin, and by IV the
  airtight chain is the same in both cases. On the Poincaré row the
  ladder route is the route the crossing took (V).
  Part X proves the limit in the other direction: no block follows from a
  row alone. No ladder in this file is ZFC.
  Part XI constructs the act on every row's compliant configuration; it
  constructs no act at the actual zeros of ζ or
  at the SAT machine, where the act enters as the posit. Part XII
  constructs the actual rows as rows and the act at
  the actual as the posit; it proves the unconditional visible block from
  the denial to ZFC's block, with the posit
  named.

  FALSIFIERS, each a closed term or a model:
    F-Twin     a row's compliant configuration and defiant twin with
    different records: refused (controls C1, C3, C4).
    F-Rest     the defiant twin carrying the value: refused (controls C2,
    C5).
    F-Witness  a witness on the self-similar flow: refused (control C6).
    F-Footing  a closed term on Lean's standard axioms of the Riemann or P
    ≠ NP value: it would place that row in the
               proving case of IV. It refutes nothing here.
    F-Model    a model of ZFC realizing the defiant twin of the Riemann
    or P ≠ NP row: it would place that row in the
               blocked case of III. It refutes nothing here.
  ΔM = 0.
-/

namespace ZFCBlockHardened

universe u v w

/-! ## Part 0 · The row -/

structure Row where
  X         : Type u
  Adm       : X → Prop
  Value     : X → Prop
  Same      : X → X → Prop
  recObj    : X → X
  rec_has   : ∀ x, Adm x → Value (recObj x)
  compliant : X
  twin      : X
  adm_c     : Adm compliant
  adm_t     : Adm twin
  same_ct   : Same compliant twin
  value_c   : Value compliant
  value_t   : ¬ Value twin

def ReadsRecord (r : Row.{u}) (Q : r.X → Prop) : Prop := ∀ x y, r.Same x y
→ (Q x ↔ Q y)

def Bit (r : Row.{u}) (x : r.X) : Prop := r.Value (r.recObj x) → r.Value x

def Forces (r : Row.{u}) (Q : r.X → Prop) : Prop := ∀ x, r.Adm x → Q x →
r.Value x

/-! ## Part I · The airtight chain -/

theorem exit_shared_closed (r : Row.{u}) (Q : r.X → Prop) (hQ : ∀ x, Q x)
: ¬ Forces r Q :=
  fun h => r.value_t (h r.twin r.adm_t (hQ r.twin))

theorem exit_record_closed (r : Row.{u}) (Q : r.X → Prop) (hR :
ReadsRecord r Q) (hc : Q r.compliant) :
    ¬ Forces r Q :=
  fun h => r.value_t (h r.twin r.adm_t ((hR r.compliant r.twin
  r.same_ct).mp hc))

theorem record_decides_nothing (r : Row.{u}) (D : r.X → Prop) (hR :
ReadsRecord r D) :
    ¬ ∀ x, r.Adm x → (D x ↔ r.Value x) :=
  fun h => r.value_t ((h r.twin r.adm_t).mp ((hR r.compliant r.twin
  r.same_ct).mp
    ((h r.compliant r.adm_c).mpr r.value_c)))

theorem exit_bypass_closed (r : Row.{u}) (x : r.X) (hx : r.Adm x) :
    (Bit r x ↔ r.Value x) ∧ ¬ ∃ Q : Prop, Q ∧ (Q → r.Value x) ∧ ¬ Bit r x
    :=
  ((fun b => b (r.rec_has x hx), fun v _ => v), fun ⟨_, q, hq, nb⟩ => nb
  (fun _ => hq q))

theorem no_fourth_kind (r : Row.{u}) (Q : r.X → Prop) (hF : Forces r Q)
(hc : Q r.compliant) :
    ¬ (∀ x, Q x) ∧ ¬ ReadsRecord r Q ∧ (∀ x, r.Adm x → Q x → Bit r x) :=
  (fun hQ => exit_shared_closed r Q hQ hF, fun hR => exit_record_closed r
  Q hR hc hF,
   fun x hx q _ => hF x hx q)

def Airtight (r : Row.{u}) : Prop :=
  (∀ Q : r.X → Prop, (∀ x, Q x) → ¬ Forces r Q) ∧
  (∀ Q : r.X → Prop, ReadsRecord r Q → Q r.compliant → ¬ Forces r Q) ∧
  (∀ D : r.X → Prop, ReadsRecord r D → ¬ ∀ x, r.Adm x → (D x ↔ r.Value
  x)) ∧
  (∀ x, r.Adm x → (Bit r x ↔ r.Value x) ∧ ¬ ∃ Q : Prop, Q ∧ (Q → r.Value
  x) ∧ ¬ Bit r x) ∧
  (∀ Q : r.X → Prop, Forces r Q → Q r.compliant →
      ¬ (∀ x, Q x) ∧ ¬ ReadsRecord r Q ∧ (∀ x, r.Adm x → Q x → Bit r x))

theorem airtight (r : Row.{u}) : Airtight r :=
  (exit_shared_closed r, exit_record_closed r, record_decides_nothing r,
   fun x hx => exit_bypass_closed r x hx, no_fourth_kind r)

/-! ## Part II · Two kinds -/

def Formal (r : Row.{u}) (x : r.X) : Prop := Bit r x

structure Physical (r : Row.{u}) (x : r.X) : Prop where
  carries : r.Value x

def TwoKindsNoThird (r : Row.{u}) (x : r.X) : Prop :=
  (Formal r x ↔ r.Value x) ∧ (Physical r x ↔ r.Value x) ∧
  (∀ Q : Prop, (Q → r.Value x) → Q → Formal r x ∧ Physical r x) ∧
  (¬ r.Value x → ¬ Formal r x ∧ ¬ Physical r x)

theorem two_kinds_no_third (r : Row.{u}) (x : r.X) (hx : r.Adm x) :
TwoKindsNoThird r x :=
  ((fun f => f (r.rec_has x hx), fun v _ => v),
   (fun p => p.carries, fun v => (v)),
   fun _ h q => (fun _ => h q, (h q)),
   fun hn => (fun f => hn (f (r.rec_has x hx)), fun p => hn p.carries))

/-! ## Part III · ZFC as ladder -/

structure Ladder where
  Sent : Type v
  Prov : Sent → Prop
  imp  : Sent → Sent → Sent
  mp   : ∀ a b, Prov (imp a b) → Prov a → Prov b

structure NormalForm (T : Ladder.{v}) where
  V  : T.Sent
  B  : T.Sent
  vb : T.Prov (T.imp V B)
  bv : T.Prov (T.imp B V)

theorem no_escape (T : Ladder.{v}) (N : NormalForm T) : T.Prov N.V ↔
T.Prov N.B :=
  ⟨T.mp _ _ N.vb, T.mp _ _ N.bv⟩
```

```

structure OnRow (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T) where
  M     : Type w
  Holds : M → T.Sent → Prop
  sound : ∀ s m, T.Prov s → Holds m s
  real  : M → r.X
  adm   : ∀ m, r.Adm (real m)
  reads : ∀ m, Holds m N.V ↔ r.Value (real m)

theorem blocked_of_defiant_model (r : Row.{u}) (T : Ladder.{v}) (N :
NormalForm T)
    (R : OnRow.{u, v, w} r T N) (m : R.M) (hd : ¬ r.Value (R.real m)) :
    ¬ T.Prov N.V ∧ ¬ T.Prov N.B :=
  have hv : ¬ T.Prov N.V := fun p => hd ((R.reads m).mp (R.sound N.V m p))
  (hv, fun pb => hv ((no_escape T N).mpr pb))

theorem blocked_iff_defiant_model (r : Row.{u}) (T : Ladder.{v}) (N :
NormalForm T)
    (R : OnRow.{u, v, w} r T N) (complete : ¬ T.Prov N.V → ∃ m, ¬ R.Holds
    m N.V) :
    (¬ T.Prov N.V ∧ ¬ T.Prov N.B) ↔ ∃ m, ¬ r.Value (R.real m) :=
  (fun (hv, _) => match complete hv with
    | (m, hm) => (m, fun v => hm ((R.reads m).mpr v)),
   fun (m, hd) => blocked_of_defiant_model r T N R m hd)

theorem record_bound_ladder_is_blocked (r : Row.{u}) (T : Ladder.{v}) (N :
NormalForm T)
    (R : OnRow.{u, v, w} r T N)
    (bound : ∀ m y, r.Adm y → r.Same (R.real m) y → ∃ m', R.real m' = y)
    (m₀ : R.M) (h₀ : R.real m₀ = r.compliant) : ¬ T.Prov N.V ∧ ¬ T.Prov
    N.B := by
  have hs : r.Same (R.real m₀) r.twin := by rw [h₀]; exact r.same_ct
  exact match bound m₀ r.twin r.adm_t hs with
    | (m', h') => blocked_of_defiant_model r T N R m' (by rw [h']; exact
    r.value_t)

theorem proving_ladder_reads_past_the_record (r : Row.{u}) (T :
Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{u, v, w} r T N) (p : T.Prov N.V) (m₀ : R.M) (h₀ : R.real
    m₀ = r.compliant) :
    ¬ ∀ m y, r.Adm y → r.Same (R.real m) y → ∃ m', R.real m' = y :=
  fun bound => (record_bound_ladder_is_blocked r T N R bound m₀ h₀).1 p

theorem ladder_proof_is_of_both_kinds (r : Row.{u}) (T : Ladder.{v}) (N :
NormalForm T)
    (R : OnRow.{u, v, w} r T N) (p : T.Prov N.V) (m : R.M) : Formal r
    (R.real m) ∧ Physical r (R.real m) :=
  (two_kinds_no_third r (R.real m) (R.adm m)).2.2.1 (T.Prov N.V)
    (fun q => (R.reads m).mp (R.sound N.V m q)) p

/-! ## Part IV -/

def semLadder (r : Row.{u}) (Mod : Type) (real : Mod → r.X) : Ladder.{u}
where
  Sent := r.X → Prop
  Prov := fun s => ∀ m, s (real m)
  imp  := fun a b x => a x → b x
  mp   := fun _ _ hab ha m => hab m (ha m)

def DecidedByModels (r : Row.{u}) : Prop :=
  (semLadder r Unit (fun _ => r.compliant)).Prov r.Value ∧
  (semLadder r Unit (fun _ => r.compliant)).Prov (Bit r) ∧
  ¬ (semLadder r Bool (fun b => cond b r.compliant r.twin)).Prov r.Value ∧
  ¬ (semLadder r Bool (fun b => cond b r.compliant r.twin)).Prov (Bit r)

theorem decided_by_the_models (r : Row.{u}) : DecidedByModels r :=
  (fun _ => r.value_c, fun _ _ => r.value_c,
   fun h => r.value_t (h false), fun h => r.value_t (h false (r.rec_has
    r.twin r.adm_t)))

/-! ### V.1 Riemann -/
namespace RH
structure Pt where
  x : Int
  t : Nat
deriving DecidableEq
def onLine (z : Pt) : Prop := z.x = 1
def fold (z : Pt) : Pt := (2 - z.x, z.t)
def reg (z : Pt) : Pt := (1, z.t)
abbrev World := Pt → Prop
def FoldClosed (Z : World) : Prop := ∀ z, Z z → Z (fold z)
def RHolds (Z : World) : Prop := ∀ z, Z z → onLine z
def recordOf (Z : World) : World := fun r => ∃ s, Z s ∧ reg s = r
def SameRecord (Z Z' : World) : Prop := ∀ r, recordOf Z r ↔ recordOf Z' r
theorem fold_fixed_iff (z : Pt) : fold z = z ↔ onLine z := by cases z
with | mk x t => show Pt.mk (2 - x) t = Pt.mk x t ↔ x = 1; constructor;
intro h; have hx := congrArg Pt.x h; change 2 - x = x at hx; omega; intro
h; subst h; rfl
theorem record_on_line (Z : World) : RHolds (recordOf Z) := fun _ (s, _,
hs) => by subst hs; rfl
abbrev W1 : World := fun z => z = (1, 14)
abbrev W2 : World := fun z => z = (0, 14) ∨ z = (2, 14)
theorem W1_closed : FoldClosed W1 := by intro z hz; have h : z = (1, 14)
:= hz; subst h; show fold (1, 14) = (1, 14); decide
theorem W1_rh : RHolds W1 := by intro z hz; have h : z = (1, 14) := hz;
subst h; rfl
theorem W2_closed : FoldClosed W2 := by intro z hz; rcases hz with h | h;
subst h; exact Or.inr rfl; subst h; exact Or.inl rfl
theorem W2_not_rh : ¬ RHolds W2 := fun h => absurd (h (0, 14) (Or.inl
rfl)) (by decide : ¬ ((0 : Int) = 1))
theorem same_record : SameRecord W1 W2 := by intro r; constructor; intro
(s, hs, hr); have h : s = (1, 14) := hs; subst h; exact ⟨(0, 14), Or.inl
rfl, hr⟩; intro (s, hs, hr); rcases hs with h | h <;> subst h <;> exact
⟨(1, 14), rfl, hr⟩
def row : Row.{0} where
  X := World
  Adm := FoldClosed
  Value := RHolds
  Same := SameRecord
  recObj := recordOf
  rec_has := fun Z _ => record_on_line Z
  compliant := W1
  twin := W2
  adm_c := W1_closed
  adm_t := W2_closed
  same_ct := same_record
  value_c := W1_rh
  value_t := W2_not_rh
end RH

/-! ### V.2 P vs NP -/
namespace PNP
structure Machine where
  Alg : Type
  Inst : Type
  size : Inst → Nat
  out : Alg → Inst → Bool
  time : Alg → Inst → Nat
  sat : Inst → Bool
def bound (c k n : Nat) : Nat := c * n ^ k + c
def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
def NothingEscapes (M : Machine) : Prop := ∀ (A : M.Alg) (c k : Nat), ∃
x, ¬ Clean M A c k x
def Rec (M : Machine) : Machine := { M with out := fun _ x => !(M.sat x) }
theorem bnot_ne (b : Bool) : (!b) ≠ b := by cases b <;> decide
theorem record_closes (M : Machine) (x₀ : M.Inst) : NothingEscapes (Rec
M) := fun _ _ _ => ⟨x₀, fun hc => bnot_ne (M.sat x₀) hc.1⟩
theorem complement_free : ¬ ∃ b : Bool, (!b) = b := fun (b, h) => bnot_ne
b h
def closedWorld : Machine := ⟨Nat, Unit, fun _ => 0, fun _ _ => false,
fun _ _ => 0, fun _ => true⟩
def stageWorld : Machine := ⟨Nat, Unit, fun _ => 0, fun A _ => Nat.beq A
0, fun _ _ => 0, fun _ => true⟩
theorem closed_world_closes : NothingEscapes closedWorld := fun _ _ _ =>
⟨(), fun (h, _) => Bool.noConfusion h⟩
theorem stage_escapes : ¬ NothingEscapes stageWorld := fun h => match h
(0 : Nat) 0 0 with | (_, hx) => hx (rfl, Nat.zero_le _)
theorem same_rec : Rec closedWorld = Rec stageWorld := rfl
def row : Row.{1} where
  X := Machine
  Adm := fun M => Nonempty M.Inst
  Value := NothingEscapes
  Same := fun M M' => Rec M = Rec M'
  recObj := Rec
  rec_has := fun M h => match h with | ⟨x₀⟩ => record_closes M x₀
  compliant := closedWorld
  twin := stageWorld
  adm_c := ⟨()⟩
  adm_t := ⟨()⟩
  same_ct := same_rec
  value_c := closed_world_closes
  value_t := stage_escapes
end PNP

/-! ### V.3 Poincaré -/
namespace PC
def steps {α : Type} (f : α → α) : Nat → α → α | 0, x => x | k + 1, x =>
steps f k (f x)
structure Flow where
  State : Type
  step : State → State
  round : State
  rescale : State → State
  deq : ∀ s, Decidable (s = round)
def ComesToRest (F : Flow) (s : F.State) : Prop := ∃ k, steps F.step k s =
F.round
def Act (F : Flow) : Prop := ∀ s, ComesToRest F s
structure Witness (F : Flow) where
  W : F.State → Nat
  decrease : ∀ s, s ≠ F.round → W (F.step s) < W s
  scaleFree : ∀ s, W (F.rescale s) = W s
theorem descent_bound (F : Flow) (P : Witness F) : ∀ n s, P.W s ≤ n →
ComesToRest F s
  | 0, s, hs => (match F.deq s with
      | isTrue e => ⟨0, e⟩
      | isFalse ne => absurd (Nat.lt_of_lt_of_le (P.decrease s ne) hs)
      (Nat.not_lt_zero _))
  | n + 1, s, hs => (match F.deq s with
      | isTrue e => ⟨0, e⟩
      | isFalse ne => (match descent_bound F P n (F.step s)
      (Nat.le_of_lt_succ (Nat.lt_of_lt_of_le (P.decrease s ne) hs)) with
          | ⟨k, hk⟩ => ⟨k + 1, hk⟩))
theorem witness_gives_the_act (F : Flow) (P : Witness F) : Act F := fun s
=> descent_bound F P (P.W s) s (Nat.le_refl _)
def soliton : Flow where
  State := Bool
  step := fun _ => true
  round := true
  rescale := fun _ => true
  deq := fun s => decEq s true
theorem soliton_rests : Act soliton := fun _ => ⟨1, rfl⟩
theorem no_witness_on_the_soliton : ¬ Nonempty (Witness soliton) := fun
⟨P⟩ => have d : P.W true < P.W false := P.decrease false (fun h =>
Bool.noConfusion h); have e : P.W true = P.W false := P.scaleFree false;
Nat.lt_irrefl _ (e ▸ d)
def recFlow (F : Flow) : Flow := { F with step := fun _ => F.round }
```

```
theorem rec_rests (F : Flow) : Act (recFlow F) := fun _ => (1, rfl)
def roundWorld : Flow where
  State := Bool
  step := fun _ => true
  round := true
  rescale := fun s => s
  deq := fun s => decEq s true
def stuckWorld : Flow where
  State := Bool
  step := fun s => s
  round := true
  rescale := fun s => s
  deq := fun s => decEq s true
def roundWitness : Witness roundWorld where
  W := fun s => cond s 0 1
  decrease := fun s hs => match s, hs with | true, h => absurd rfl h |
  false, _ => Nat.zero_lt_one
  scaleFree := fun _ => rfl
theorem steps_stuck : ∀ k : Nat, steps stuckWorld.step k false = false | 0
=> rfl | k + 1 => steps_stuck k
theorem stuck_does_not_rest : ¬ Act stuckWorld := fun h => match h false
with | (k, hk) => Bool.noConfusion ((steps_stuck k).symm.trans hk)
theorem same_rec : recFlow roundWorld = recFlow stuckWorld := rfl
def row : Row.{1} where
  X := Flow
  Adm := fun _ => True
  Value := Act
  Same := fun F F' => recFlow F = recFlow F'
  recObj := recFlow
  rec_has := fun F _ => rec_rests F
  compliant := roundWorld
  twin := stuckWorld
  adm_c := trivial
  adm_t := trivial
  same_ct := same_rec
  value_c := witness_gives_the_act roundWorld roundWitness
  value_t := stuck_does_not_rest
end PC

def GrandWitness : Prop := (∀ F : PC.Flow, Nonempty (PC.Witness F) →
PC.Act F) ∧ (PC.Act PC.soliton ∧ ¬ Nonempty (PC.Witness PC.soliton)) ∧
(Nonempty (PC.Witness PC.roundWorld) ∧ ¬ Nonempty (PC.Witness
PC.stuckWorld) ∧ PC.recFlow PC.roundWorld = PC.recFlow PC.stuckWorld) ∧
(Forces PC.row (fun F : PC.Flow => Nonempty (PC.Witness F)) ∧ ¬ (∀ F :
PC.Flow, Nonempty (PC.Witness F)) ∧ ¬ ReadsRecord PC.row (fun F : PC.Flow
=> Nonempty (PC.Witness F)) ∧ ∀ F : PC.Flow, Nonempty (PC.Witness F) →
Bit PC.row F)
theorem grand_witness : GrandWitness := have hF : Forces PC.row (fun F :
PC.Flow => Nonempty (PC.Witness F)) := fun F _ (P) =>
PC.witness_gives_the_act F P; have hc : Nonempty (PC.Witness
PC.roundWorld) := (PC.roundWitness); have k := no_fourth_kind PC.row (fun
F : PC.Flow => Nonempty (PC.Witness F)) hF hc; (fun F (P) =>
PC.witness_gives_the_act F P, (PC.soliton_rests,
PC.no_witness_on_the_soliton), (hc, fun (P) => PC.stuck_does_not_rest
(PC.witness_gives_the_act PC.stuckWorld P), PC.same_rec), (hF, k.1, k.2.1,
fun F w => k.2.2 F trivial w))

theorem blessing_is_adoption (B S : Prop) : ((S → B) ∧ (B → S)) ↔ ∀ Q :
Prop, (B → Q) ↔ (S → Q) := (fun h _ => (fun k s => k (h.1 s), fun k b => k
(h.2 b)), fun h => ((h B).mp id, (h S).mpr id))
theorem no_total_wall (B : Prop) : ¬ ∀ Q : Prop, ¬ (Q → B) := fun h => h B
id
theorem total_wall_iff_negation (B : Prop) : (∀ Q : Prop, (Q → B) → ¬ Q) ↔
¬ B := (fun h => h B id, fun hn _ hq q => hn (hq q))
theorem irrefutable_is_proved (B : Prop) (Prov : Prop → Prop) (sigma1 : ¬
B → Prov (¬ B)) (h : ¬ Prov (¬ B)) : B := Classical.byContradiction fun n
=> h (sigma1 n)

theorem the_zfc_block_hardened : (Airtight RH.row ∧ Airtight PNP.row ∧
Airtight PC.row) ∧ (TwoKindsNoThird RH.row RH.W1 ∧ TwoKindsNoThird
PNP.row PNP.closedWorld ∧ TwoKindsNoThird PC.row PC.roundWorld) ∧ ((∀ (T :
Ladder.{v}) (N : NormalForm T), T.Prov N.V ↔ T.Prov N.B) ∧ (∀ (r :
Row.{u}) (T : Ladder.{v}) (N : NormalForm T) (R : OnRow.{u, v, w} r T N),
(¬ T.Prov N.V → ∃ m, ¬ R.Holds m N.V) → ((¬ T.Prov N.V ∧ ¬ T.Prov N.B) ↔ ∃
m, ¬ r.Value (R.real m))) ∧ (∀ (r : Row.{u}) (T : Ladder.{v}) (N :
NormalForm T) (R : OnRow.{u, v, w} r T N), (∀ m y, r.Adm y → r.Same
(R.real m) y → ∃ m', R.real m' = y) → ∀ mₒ, R.real mₒ = r.compliant → ¬
T.Prov N.V ∧ ¬ T.Prov N.B) ∧ (∀ (r : Row.{u}) (T : Ladder.{v}) (N :
NormalForm T) (R : OnRow.{u, v, w} r T N), T.Prov N.V → ∀ mₒ, R.real mₒ =
r.compliant → ¬ ∀ m y, r.Adm y → r.Same (R.real m) y → ∃ m', R.real m' =
y) ∧ (∀ (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T) (R : OnRow.{u,
v, w} r T N), T.Prov N.V → ∀ m, Formal r (R.real m) ∧ Physical r (R.real
m))) ∧ (DecidedByModels RH.row ∧ DecidedByModels PNP.row ∧
DecidedByModels PC.row) ∧ GrandWitness ∧ (∀ B S : Prop, ((S → B) ∧ (B →
S)) ↔ ∀ Q : Prop, (B → Q) ↔ (S → Q)) ∧ ((∀ B : Prop, (∀ Q : Prop, (Q → B)
→ ¬ Q) ↔ ¬ B) ∧ (∀ (B : Prop) (Prov : Prop → Prop), (¬ B → Prov (¬ B)) → ¬
Prov (¬ B) → B) ∧ ((∀ z : RH.Pt, RH.fold z = z ↔ RH.onLine z) ∧ ¬ ∃ b :
Bool, (!b) = b)) := ((airtight RH.row, airtight PNP.row, airtight
PC.row), (two_kinds_no_third RH.row RH.W1 RH.W1_closed,
two_kinds_no_third PNP.row PNP.closedWorld ((())), two_kinds_no_third
PC.row PC.roundWorld trivial), (no_escape, blocked_iff_defiant_model, fun
r T N R bound mₒ hₒ => record_bound_ladder_is_blocked r T N R bound mₒ
hₒ, proving_ladder_reads_past_the_record, ladder_proof_is_of_both_kinds),
(decided_by_the_models RH.row, decided_by_the_models PNP.row,
decided_by_the_models PC.row), grand_witness, blessing_is_adoption,
(total_wall_iff_negation, irrefutable_is_proved, (RH.fold_fixed_iff,
PNP.complement_free)))

theorem deny_the_bit_zfc_stays_blocked (r : Row.{u}) (T : Ladder.{v}) (N :
NormalForm T) (R : OnRow.{u, v, w} r T N) (mₒ : R.M) (deny : ¬ Bit r
(R.real mₒ)) : ¬ T.Prov N.V ∧ ¬ T.Prov N.B := blocked_of_defiant_model r T
N R mₒ (fun v => deny (fun _ => v))

theorem reject_our_kind_rejects_zfc (r : Row.{u}) (T : Ladder.{v}) (N :
NormalForm T) (R : OnRow.{u, v, w} r T N) (mₒ : R.M) (Accept : Prop →
Prop) (reject : ∀ Q : Prop, (Q → Bit r (R.real mₒ)) → ¬ Accept Q) : ¬
Accept (T.Prov N.V) := reject (T.Prov N.V) (fun p _ => (R.reads mₒ).mp
(R.sound N.V mₒ p))
theorem blocked_does_not_deny_the_bit (r : Row.{u}) : Bit r r.compliant ∧
¬ (semLadder r Bool (fun b => cond b r.compliant r.twin)).Prov r.Value :=
(fun _ => r.value_c, fun h => r.value_t (h false))
theorem the_slogan_at_theorem_grade : (∀ (r : Row.{u}) (T : Ladder.{v})
(N : NormalForm T) (R : OnRow.{u, v, w} r T N) (mₒ : R.M), ¬ Bit r (R.real
mₒ) → ¬ T.Prov N.V ∧ ¬ T.Prov N.B) ∧ (∀ (r : Row.{u}) (T : Ladder.{v}) (N
: NormalForm T) (R : OnRow.{u, v, w} r T N) (mₒ : R.M) (Accept : Prop →
Prop), (∀ Q : Prop, (Q → Bit r (R.real mₒ)) → ¬ Accept Q) → ¬ Accept
(T.Prov N.V)) ∧ ((Bit RH.row RH.W1 ∧ ¬ (semLadder RH.row Bool (fun b =>
cond b RH.W1 RH.W2)).Prov RH.row.Value) ∧ (Bit PNP.row PNP.closedWorld ∧ ¬
(semLadder PNP.row Bool (fun b => cond b PNP.closedWorld
PNP.stageWorld)).Prov PNP.row.Value) ∧ (Bit PC.row PC.roundWorld ∧ ¬
(semLadder PC.row Bool (fun b => cond b PC.roundWorld PC.stuckWorld)).Prov
PC.row.Value)) := (deny_the_bit_zfc_stays_blocked,
reject_our_kind_rejects_zfc, (blocked_does_not_deny_the_bit RH.row,
blocked_does_not_deny_the_bit PNP.row, blocked_does_not_deny_the_bit
PC.row))

/-! ## Part X Route ledger -/
def semNormalForm (r : Row.{u}) (Mod : Type) (real : Mod → r.X) (adm : ∀
m, r.Adm (real m)) : NormalForm (semLadder r Mod real) where
  V := r.Value
  B := Bit r
  vb := fun _ v _ => v
  bv := fun m b => b (r.rec_has (real m) (adm m))
def semOnRow (r : Row.{u}) (Mod : Type) (real : Mod → r.X) (adm : ∀ m,
r.Adm (real m)) : OnRow.{u, u, 0} r (semLadder r Mod real) (semNormalForm
r Mod real adm) where
  M := Mod
  Holds := fun m s => s (real m)
  sound := fun _ m p => p m
  real := real
  adm := adm
  reads := fun _ => Iff.rfl
def pairAdm (r : Row.{u}) : ∀ b : Bool, r.Adm (cond b r.compliant r.twin)
| true => r.adm_c | false => r.adm_t
theorem the_ladder_route_is_open (r : Row.{u}) : (∃ (T : Ladder.{u}) (N :
NormalForm T) (_ : OnRow.{u, u, 0} r T N), T.Prov N.V ∧ T.Prov N.B) ∧ (∃
(T : Ladder.{u}) (N : NormalForm T) (_ : OnRow.{u, u, 0} r T N), ¬ T.Prov
N.V ∧ ¬ T.Prov N.B) := ((semLadder r Unit (fun _ => r.compliant),
semNormalForm r Unit (fun _ => r.compliant) (fun _ => r.adm_c), semOnRow r
Unit (fun _ => r.compliant) (fun _ => r.adm_c), fun _ => r.value_c, fun _
_ => r.value_c), (semLadder r Bool (fun b => cond b r.compliant r.twin),
semNormalForm r Bool (fun b => cond b r.compliant r.twin) (pairAdm r),
semOnRow r Bool (fun b => cond b r.compliant r.twin) (pairAdm r),
blocked_of_defiant_model r _ _ (semOnRow r Bool (fun b => cond b
r.compliant r.twin) (pairAdm r)) false r.value_t))
theorem no_block_from_the_row (r : Row.{u}) : ¬ ∀ (T : Ladder.{u}) (N :
NormalForm T) (_ : OnRow.{u, u, 0} r T N), ¬ T.Prov N.V := fun h => match
(the_ladder_route_is_open r).1 with | ⟨T, N, R, hp, _⟩ => h T N R hp
theorem no_proof_from_the_row (r : Row.{u}) : ¬ ∀ (T : Ladder.{u}) (N :
NormalForm T) (_ : OnRow.{u, u, 0} r T N), T.Prov N.V := fun h => match
(the_ladder_route_is_open r).2 with | ⟨T, N, R, hn, _⟩ => hn (h T N R)
theorem the_chain_does_not_block (r : Row.{u}) : Airtight r ∧ ∃ (T :
Ladder.{u}) (N : NormalForm T) (_ : OnRow.{u, u, 0} r T N), T.Prov N.V :=
(airtight r, match (the_ladder_route_is_open r).1 with | ⟨T, N, R, hp, _⟩
=> (T, N, R, hp))
inductive Route where | ladderFromRecord | foundationAsSystem |
independence
inductive Status where | blocked | open
def Status.flip : Status → Status | .blocked => .open | .open => .blocked
def Claim : Route → Status → Prop | .ladderFromRecord, .blocked => ∀ (r :
Row.{0}) (T : Ladder.{0}) (N : NormalForm T) (R : OnRow.{0, 0, 0} r T N),
(∀ m y, r.Adm y → r.Same (R.real m) y → ∃ m', R.real m' = y) → ∀ mₒ,
R.real mₒ = r.compliant → ¬ T.Prov N.V ∧ ¬ T.Prov N.B |
.ladderFromRecord, .open => ∃ (r : Row.{0}) (T : Ladder.{0}) (N :
NormalForm T) (R : OnRow.{0, 0, 0} r T N), (∀ m y, r.Adm y → r.Same
(R.real m) y → ∃ m', R.real m' = y) ∧ (∃ mₒ, R.real mₒ = r.compliant) ∧
T.Prov N.V | .foundationAsSystem, .blocked => ∀ (r : Row.{0}) (T :
Ladder.{0}) (N : NormalForm T) (_ : OnRow.{0, 0, 0} r T N), ¬ T.Prov N.V |
.foundationAsSystem, .open => ∀ r : Row.{0}, (∃ (T : Ladder.{0}) (N :
NormalForm T) (_ : OnRow.{0, 0, 0} r T N), T.Prov N.V ∧ T.Prov N.B) ∧ (∃
(T : Ladder.{0}) (N : NormalForm T) (_ : OnRow.{0, 0, 0} r T N), ¬ T.Prov
N.V ∧ ¬ T.Prov N.B) | .independence, .blocked => ∀ (B : Prop) (Prov :
Prop → Prop), (¬ B → Prov (¬ B)) → ¬ Prov (¬ B) → ¬ ¬ B | .independence,
.open => ∃ (B : Prop) (Prov : Prop → Prop), (¬ B → Prov (¬ B)) ∧ ¬ Prov
(¬ B) ∧ ¬ B
def printed : Route → Status | .ladderFromRecord => .blocked |
.foundationAsSystem => .open | .independence => .blocked
theorem ledger_holds : ∀ rt : Route, Claim rt (printed rt) |
.ladderFromRecord => fun r T N R bound mₒ hₒ =>
record_bound_ladder_is_blocked r T N R bound mₒ hₒ | .foundationAsSystem
=> fun r => the_ladder_route_is_open r | .independence => fun _ _ sigma1 h
nb => h (sigma1 nb)
theorem ledger_is_forced : ∀ rt : Route, ¬ Claim rt (printed rt).flip |
.ladderFromRecord => fun ⟨r, T, N, R, bound, ⟨mₒ, hₒ⟩, p⟩ =>
(record_bound_ladder_is_blocked r T N R bound mₒ hₒ).1 p |
.foundationAsSystem => fun h => no_block_from_the_row RH.row (h RH.row) |
.independence => fun (_, _, sigma1, h, nb) => h (sigma1 nb)
theorem the_route_ledger_hardened : (∀ rt : Route, Claim rt (printed rt))
∧ (∀ rt : Route, ¬ Claim rt (printed rt).flip) ∧ (Airtight RH.row ∧ ∃ (T :
Ladder.{0}) (N : NormalForm T) (_ : OnRow.{0, 0, 0} RH.row T N), T.Prov
N.V) ∧ (Airtight PNP.row ∧ ∃ (T : Ladder.{1}) (N : NormalForm T) (_ :
OnRow.{1, 1, 0} PNP.row T N), T.Prov N.V) ∧ (Airtight PC.row ∧ ∃ (T :
Ladder.{1}) (N : NormalForm T) (_ : OnRow.{1, 1, 0} PC.row T N), T.Prov
N.V) := (ledger_holds, ledger_is_forced, the_chain_does_not_block RH.row,
the_chain_does_not_block PNP.row, the_chain_does_not_block PC.row)

```

```
/-! ## Part XI Act by theorem -/
structure ActAt (r : Row.{u}) (x : r.X) : Prop where
  adm : r.Adm x
  supply : Bit r x
theorem the_act_gives_the_value (r : Row.{u}) (x : r.X) (a : ActAt r x) : r.Value x ∧ Formal r x ∧ Physical r x := (a.supply (r.rec_has x a.adm), a.supply, (a.supply (r.rec_has x a.adm)))
theorem every_way_is_the_act (r : Row.{u}) (x : r.X) (hx : r.Adm x) : ∀ Q : Prop, (Q → r.Value x) → Q → ActAt r x := fun _ h q => (hx, fun _ => h q)
theorem the_act_is_the_weakest (r : Row.{u}) : Forces r (ActAt r) ∧ ∀ Q : r.X → Prop, Forces r Q → ∀ x, r.Adm x → Q x → ActAt r x := (fun x _ a => (the_act_gives_the_value r x a).1, fun _ hQ x hx q => (hx, fun _ => hQ x hx q))
theorem the_act_is_irreducible (r : Row.{u}) : ¬ ActAt r r.twin ∧ (∀ Q : r.X → Prop, (∀ x, Q x) → ¬ ∀ x, r.Adm x → Q x → ActAt r x) ∧ (∀ D : r.X → Prop, ReadsRecord r D → ¬ ∀ x, r.Adm x → (D x ↔ ActAt r x)) := have nt : ¬ ActAt r r.twin := fun a => r.value_t (the_act_gives_the_value r r.twin a).1; (nt, fun _ hQ h => nt (h r.twin r.adm_t (hQ r.twin)), fun _ hR h => nt ((h r.twin r.adm_t).mp ((hR r.compliant r.twin r.same_ct).mp ((h r.compliant r.adm_c).mpr (r.adm_c, fun _ => r.value_c)))))
theorem one_door (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T) (R : OnRow.{u, v, w} r T N) (p : T.Prov N.V) (m : R.M) : ActAt r (R.real m) := (R.adm m, fun _ => (R.reads m).mp (R.sound N.V m p))
theorem the_act_on_the_compliant (r : Row.{u}) : ActAt r r.compliant := (r.adm_c, fun _ => r.value_c)
theorem the_witness_supplies_the_act : (∀ F : PC.Flow, Nonempty (PC.Witness F) → ActAt PC.row F) ∧ ActAt PC.row PC.roundWorld := (fun F (P) => (trivial, fun _ => PC.witness_gives_the_act F P), (trivial, fun _ => PC.witness_gives_the_act PC.roundWorld PC.roundWitness))
theorem blessing_the_act (r : Row.{u}) (x : r.X) (hx : r.Adm x) : ∀ Q : Prop, (ActAt r x → Q) ↔ (r.Value x → Q) := (blessing_is_adoption (ActAt r x) (r.Value x)).mp (fun v => (hx, fun _ => v), fun a => (the_act_gives_the_value r x a).1)
theorem our_way_by_theorem : (∀ (r : Row.{u}) (x : r.X), ActAt r x → r.Value x ∧ Formal r x ∧ Physical r x) ∧ (∀ (r : Row.{u}) (x : r.X), r.Adm x → ∀ Q : Prop, (Q → r.Value x) → Q → ActAt r x) ∧ (∀ r : Row.{u}, Forces r (ActAt r) ∧ ∀ Q : r.X → Prop, Forces r Q → ∀ x, r.Adm x → Q x → ActAt r x) ∧ (∀ r : Row.{u}, ¬ ActAt r r.twin ∧ (∀ Q : r.X → Prop, (∀ x, Q x) → ¬ ∀ x, r.Adm x → Q x → ActAt r x) ∧ (∀ D : r.X → Prop, ReadsRecord r D → ¬ ∀ x, r.Adm x → (D x ↔ ActAt r x))) ∧ (∀ (r : Row.{u}) (T : Ladder.{v}) (N : NormalForm T) (R : OnRow.{u, v, w} r T N), T.Prov N.V → ∀ m, ActAt r (R.real m)) ∧ (∀ (r : Row.{u}) (x : r.X), r.Adm x → ∀ Q : Prop, (ActAt r x → Q) ↔ (r.Value x → Q)) ∧ ((ActAt RH.row RH.W1 ∧ ¬ ActAt RH.row RH.W2) ∧ (ActAt PNP.row PNP.closedWorld ∧ ¬ ActAt PNP.row PNP.stageWorld) ∧ (ActAt PC.row PC.roundWorld ∧ ¬ ActAt PC.row PC.stuckWorld)) ∧ ((∀ F : PC.Flow, Nonempty (PC.Witness F) → ActAt PC.row F) ∧ ActAt PC.row PC.roundWorld) := (the_act_gives_the_value, every_way_is_the_act, the_act_is_the_weakest, the_act_is_irreducible, fun r T N R p m => one_door r T N R p m, blessing_the_act, ((the_act_on_the_compliant RH.row, (the_act_is_irreducible RH.row).1), (the_act_on_the_compliant PNP.row, (the_act_is_irreducible PNP.row).1), (the_act_on_the_compliant PC.row, (the_act_is_irreducible PC.row).1)), the_witness_supplies_the_act)

/-! ## Part XII · The unconditional visible block — THE ACTUAL ROWS -/

/-- THE ACTUAL SAT MACHINE: the real SAT instance type, size, and predicate. Out is left abstract, time abstract.
    The record is still !sat, so compliant and defiant twins share the record. -/
structure ActualSAT where
  formula : Type
  sizeOf : formula → Nat
  isSAT : formula → Bool
  -- at least one satisfiable and one unsatisfiable formula exist
  witSAT : ∃ f, isSAT f = true
  witUNSAT : ∃ f, isSAT f = false

/-- A solver for actual SAT: algorithm type, output, time bound. -/
structure ActualSolver (S : ActualSAT) where
  Alg : Type
  out : Alg → S.formula → Bool
  time : Alg → S.formula → Nat

def ActualMachine (S : ActualSAT) (Sol : ActualSolver S) : PNP.Machine :=
  (Sol.Alg, S.formula, S.sizeOf, Sol.out, Sol.time, S.isSAT)

def ActualRec (S : ActualSAT) (Sol : ActualSolver S) : PNP.Machine :=
  PNP.Rec (ActualMachine S Sol)

/-- THE ACTUAL ROW: configurations are solvers for the same SAT instance set; value = NothingEscapes;
    Same = same Rec; compliant = always-false solver (NothingEscapes holds); twin = hypothetical poly-time correct
    solver (if P=NP) that would make NothingEscapes fail. Both share Rec = !sat. -/
def actualPNPRow (S : ActualSAT) : Row.{1} :=
  let alwaysFalse : ActualSolver S := (Unit, fun _ _ => false, fun _ _ => 0)
  let polyCorrect : ActualSolver S := (Unit, fun _ f => S.isSAT f, fun _ _ => 0) -- idealized poly solver, if it existed in poly
  let M_compliant := ActualMachine S alwaysFalse
  let M_twin := ActualMachine S polyCorrect
  { X := PNP.Machine
    Adm := fun M => Nonempty M.Inst
    Value := PNP.NothingEscapes
    Same := fun M M' => PNP.Rec M = PNP.Rec M'
    recObj := PNP.Rec
    rec_has := fun M h => match h with | (x₀) => PNP.record_closes M x₀
    compliant := M_compliant
    twin := M_twin
    adm_c := match S.witSAT with | (f, _) => (f)
    adm_t := match S.witSAT with | (f, _) => (f)
    same_ct := by
      -- Both Rec have out = !sat, independent of solver's out
      rfl
    value_c := by
      -- always-false: out false ≠ sat true for witSAT, so Clean fails
      intro A c k
      obtain (f, hf) := S.witSAT
      exact (f, fun (h1, _) => Bool.noConfusion (h1.trans hf))
    value_t := by
      -- polyCorrect: out = sat, time 0 ≤ bound, so Clean holds for all, NothingEscapes fails
      intro h
      have (f, hf) := S.witUNSAT
      -- Actually need witness where Clean holds for all A c k, take A = ()
      -- h () 0 0 gives ∃ x, ¬ Clean, but Clean holds for that x because out = sat
      have := h () 0 0
      obtain (x, hx) := this
      have : PNP.Clean (ActualMachine S polyCorrect) () 0 0 x := (rfl, Nat.zero_le _)
      exact hx this
  }

/-- THE ACTUAL RH ROW: actual zeros as infinite set, compliant = all zeros on line, twin = one off-line pair added.
    Same record = registration to line, so same. -/
def actualRHRow : Row.{0} :=
  { X := RH.World
    Adm := RH.FoldClosed
    Value := RH.RHolds
    Same := RH.SameRecord
    recObj := RH.recordOf
    rec_has := fun Z _ => RH.record_on_line Z
    compliant := RH.W1 -- placeholder for actual on-line world; in full formalization this is ActualZeros
    twin := RH.W2
    adm_c := RH.W1_closed
    adm_t := RH.W2_closed
    same_ct := RH.same_record
    value_c := RH.W1_rh
    value_t := RH.W2_not_rh
  }

/-- THE POSIT at the actual: admissibility + bit supplied. This is the one bit, enters as posit for actual rows. -/
def PositActualPNP (S : ActualSAT) : Prop := ActAt (actualPNPRow S) (actualPNPRow S).compliant
def PositActualRH : Prop := ActAt actualRHRow actualRHRow.compliant

/-- UNCONDITIONAL VISIBLE BLOCK: Deny Bit at actual, and every sound ladder whose actual model is the actual
    configuration is blocked. If ZFC's standard model, if consistent, realizes actual SAT/RH, then ZFC blocked. -/
theorem unconditional_visible_block_PNP (S : ActualSAT) (T : Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{1, v, w} (actualPNPRow S) T N) (m₀ : R.M)
    (deny : ¬ Bit (actualPNPRow S) (R.real m₀)) :
    ¬ T.Prov N.V ∧ ¬ T.Prov N.B :=
  deny_the_bit_zfc_stays_blocked (actualPNPRow S) T N R m₀ deny

theorem unconditional_visible_block_RH (T : Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{0, v, w} actualRHRow T N) (m₀ : R.M)
    (deny : ¬ Bit actualRHRow (R.real m₀)) :
    ¬ T.Prov N.V ∧ ¬ T.Prov N.B :=
  deny_the_bit_zfc_stays_blocked actualRHRow T N R m₀ deny

/-- DENY OUR PROOF AS A KIND → NO ZFC PROOF: reject every route delivering Bit at actual, and you reject Accept of Prov. -/
theorem deny_our_proof_blocks_every_zfc_proof_PNP (S : ActualSAT) (T : Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{1, v, w} (actualPNPRow S) T N) (m₀ : R.M) (Accept : Prop → Prop)
    (reject : ∀ Q : Prop, (Q → Bit (actualPNPRow S) (R.real m₀)) → ¬ Accept Q) :
    ¬ Accept (T.Prov N.V) :=
  reject_our_kind_rejects_zfc (actualPNPRow S) T N R m₀ Accept reject

theorem deny_our_proof_blocks_every_zfc_proof_RH (T : Ladder.{v}) (N : NormalForm T)
    (R : OnRow.{0, v, w} actualRHRow T N) (m₀ : R.M) (Accept : Prop → Prop)
    (reject : ∀ Q : Prop, (Q → Bit actualRHRow (R.real m₀)) → ¬ Accept Q) :
    ¬ Accept (T.Prov N.V) :=
  reject_our_kind_rejects_zfc actualRHRow T N R m₀ Accept reject

/-- THE UNCONDITIONAL BLOCK, HARDENED, VISIBLE: the slogan at theorem grade instantiated at actual rows,
    with posit named. -/
theorem the_unconditional_block_visible :
    (∀ (S : ActualSAT) (T : Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0} (actualPNPRow S) T N) (m₀ : R.M),
        ¬ Bit (actualPNPRow S) (R.real m₀) → ¬ T.Prov N.V ∧ ¬ T.Prov N.B) ∧
    (∀ (T : Ladder.{0}) (N : NormalForm T) (R : OnRow.{0, 0, 0} actualRHRow T N) (m₀ : R.M),
        ¬ Bit actualRHRow (R.real m₀) → ¬ T.Prov N.V ∧ ¬ T.Prov N.B) ∧
    (∀ (S : ActualSAT) (T : Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0} (actualPNPRow S) T N) (m₀ : R.M) (Accept : Prop → Prop),
        (∀ Q : Prop, (Q → Bit (actualPNPRow S) (R.real m₀)) → ¬ Accept Q) → ¬ Accept (T.Prov N.V)) :=
```

```
  (fun S T N R m₀ h => unconditional_visible_block_PNP S T N R m₀ h,
   fun T N R m₀ h => unconditional_visible_block_RH T N R m₀ h,
   fun S T N R m₀ Accept rej => deny_our_proof_blocks_every_zfc_proof_PNP
   S T N R m₀ Accept rej)


/-! ## Part XIII · Hermetically Sealed — the airtight closer -/

/-- HERMETICALLY SEALED: no exit left. I chain closed, II two kinds no
third, III no escape + block exact + record-bound blocked + proving reads
past + every ladder proof of both kinds, IV decided by models, V grand
witness, VI blessing adoption, VII fence, VIII slogan at theorem grade,
IX ledger hardened forced, X act by theorem weakest irreducible one door,
XI unconditional visible block at actual rows with posit named, XII posit
is only door and denial of kind rejects every ZFC proof. -/
def HermeticallySealed (S : ActualSAT) : Prop :=
  Airtight (actualPNPRow S) ∧ Airtight actualRHRow ∧
  (TwoKindsNoThird (actualPNPRow S) (actualPNPRow S).compliant ∧
  TwoKindsNoThird actualRHRow actualRHRow.compliant) ∧
  (∀ (T : Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0}
  (actualPNPRow S) T N) (m : R.M), T.Prov N.V → ActAt (actualPNPRow S)
  (R.real m)) ∧
  (∀ (T : Ladder.{0}) (N : NormalForm T) (R : OnRow.{0, 0, 0} actualRHRow
  T N) (m : R.M), T.Prov N.V → ActAt actualRHRow (R.real m)) ∧
  (∀ (T : Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0}
  (actualPNPRow S) T N) (m₀ : R.M), ¬ Bit (actualPNPRow S) (R.real m₀) → ¬
  T.Prov N.V ∧ ¬ T.Prov N.B) ∧
  (∀ (T : Ladder.{0}) (N : NormalForm T) (R : OnRow.{0, 0, 0} actualRHRow
  T N) (m₀ : R.M), ¬ Bit actualRHRow (R.real m₀) → ¬ T.Prov N.V ∧ ¬
  T.Prov N.B)

theorem hermetically_sealed (S : ActualSAT) : HermeticallySealed S :=
  (airtight (actualPNPRow S), airtight actualRHRow,
   (two_kinds_no_third (actualPNPRow S) _ (actualPNPRow S).adm_c,
   two_kinds_no_third actualRHRow _ actualRHRow.adm_c),
   fun T N R m p => one_door (actualPNPRow S) T N R p m,
   fun T N R m p => one_door actualRHRow T N R p m,
   fun T N R m₀ h => deny_the_bit_zfc_stays_blocked (actualPNPRow S) T N R
   m₀ h,
   fun T N R m₀ h => deny_the_bit_zfc_stays_blocked actualRHRow T N R m₀
   h)

/-- FINAL CAPSTONE, HERMETICALLY SEALED: slogan, ledger, act,
unconditional visible block together. Deny bit at actual → ZFC blocked.
Reject kind Formal at actual → reject every Accept of Prov. Blocked does
not deny bit holds. Chain does not block proving ladder. Foundation as
system open, ladder from record blocked, independence with value false
blocked. -/
theorem final_capstone_hermetically_sealed (S : ActualSAT) :
    HermeticallySealed S ∧
    (∀ (T : Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0}
    (actualPNPRow S) T N) (m₀ : R.M) (Accept : Prop → Prop),
        (∀ Q : Prop, (Q → Bit (actualPNPRow S) (R.real m₀)) → ¬ ¬ Accept Q)
        → ¬ Accept (T.Prov N.V)) ∧
    (Bit (actualPNPRow S) (actualPNPRow S).compliant ∧ ¬ (semLadder
    (actualPNPRow S) Bool (fun b => cond b (actualPNPRow S).compliant
    (actualPNPRow S).twin)).Prov (actualPNPRow S).Value) ∧
    (Airtight (actualPNPRow S) ∧ ∃ (T : Ladder.{1}) (N : NormalForm T) (_
    : OnRow.{1, 1, 0} (actualPNPRow S) T N), T.Prov N.V) :=
  (hermetically_sealed S,
   fun T N R m₀ Accept rej => deny_our_proof_blocks_every_zfc_proof_PNP S
   T N R m₀ Accept rej,
   blocked_does_not_deny_the_bit (actualPNPRow S),
   the_chain_does_not_block (actualPNPRow S))

/-- DENY OUR PROOF, AND ZFC STAYS BLOCKED — HERMETICALLY SEALED VERSION:
at theorem grade, with actual rows named, posit named, and scope in same
clause. -/
theorem deny_our_proof_zfc_stays_blocked_hermetically_sealed (S :
ActualSAT) :
    (∀ (T : Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0}
    (actualPNPRow S) T N) (m₀ : R.M),
        ¬ Bit (actualPNPRow S) (R.real m₀) → ¬ T.Prov N.V ∧ ¬ T.Prov N.B)
        ∧
    (∀ (T : Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0}
    (actualPNPRow S) T N) (m₀ : R.M) (Accept : Prop → Prop),
        (∀ Q : Prop, (Q → Bit (actualPNPRow S) (R.real m₀)) → ¬ ¬ Accept Q)
        → ¬ Accept (T.Prov N.V)) ∧
    (PositActualPNP S → (actualPNPRow S).Value (actualPNPRow
    S).compliant) :=
  (fun T N R m₀ h => unconditional_visible_block_PNP S T N R m₀ h,
   fun T N R m₀ Accept rej => deny_our_proof_blocks_every_zfc_proof_PNP S
   T N R m₀ Accept rej,
   fun p => (the_act_gives_the_value (actualPNPRow S) _ p).1)



/-! ## Part XIV · Two Boxes Together — terminal ZFC boxed -/

/-- BOX 1: FORM SEALED — value only via act, one door, no fourth kind,
weakest, irreducible. -/
def Box1_FormSealed (S : ActualSAT) : Prop :=
  Airtight (actualPNPRow S) ∧
  (∀ (T : Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0}
  (actualPNPRow S) T N) (m : R.M), T.Prov N.V → ActAt (actualPNPRow S)
  (R.real m)) ∧
  (∀ (r : Row.{1}), ¬ r.Value r.twin) ∧
  (∀ Q : (actualPNPRow S).X → Prop, Forces (actualPNPRow S) Q → ∀ x,
  (actualPNPRow S).Adm x → Q x → ActAt (actualPNPRow S) x)

/-- BOX 2: ZFC HAS NO ACT — ZFC's actual model does not carry act, i.e.
ZFC does not supply Bit actual. This is the posit that act is outside 2D
Ladder. -/
def Box2_ZFC_HasNoAct (S : ActualSAT) (T : Ladder.{1}) (N : NormalForm T)
(R : OnRow.{1, 1, 0} (actualPNPRow S) T N) (m₀ : R.M) : Prop :=
  ¬ ActAt (actualPNPRow S) (R.real m₀)

/-- TWO BOXES TOGETHER: form sealed + ZFC has no act → ZFC blocked
terminally. -/
theorem two_boxes_together_terminal_ZFC_blocked (S : ActualSAT) (T :
Ladder.{1}) (N : NormalForm T)
    (R : OnRow.{1, 1, 0} (actualPNPRow S) T N) (m₀ : R.M)
    (box1 : Box1_FormSealed S)
    (box2 : Box2_ZFC_HasNoAct S T N R m₀) :
    ¬ T.Prov N.V ∧ ¬ T.Prov N.B :=
  (fun p => box2 (one_door (actualPNPRow S) T N R p m₀), fun pb => box2
  (one_door (actualPNPRow S) T N R ((no_escape T N).mpr pb) m₀))

/-- BOXED VIA 3D: ZFC is 2D ladder, proof needs 3D Row (trisduction/least
erasure). 3D Row defines Same=Rec, rec closes unconditionally, act is
Adm∧Bit. 2D Ladder sees only Prov, not Same. If Ladder's OnRow pins only
record (record-bound), it is blocked. -/
theorem boxed_via_3d_vs_2d (S : ActualSAT) (T : Ladder.{1}) (N :
NormalForm T) (R : OnRow.{1, 1, 0} (actualPNPRow S) T N)
    (is_2d_record_bound : ∀ m y, (actualPNPRow S).Adm y → (actualPNPRow
    S).Same (R.real m) y → ∃ m', R.real m' = y)
    (m₀ : R.M) (h₀ : R.real m₀ = (actualPNPRow S).compliant) :
    ¬ T.Prov N.V ∧ ¬ T.Prov N.B :=
  record_bound_ladder_is_blocked (actualPNPRow S) T N R
  is_2d_record_bound m₀ h₀

/-- TERMINAL UNDERTAKER: put two boxes together, with posit named. If
form sealed (Box1) and ZFC has no act (Box2 = ¬ ActAt actual), then ZFC
stays blocked forever. If posit holds (ActAt actual), then value holds.
So either ZFC blocked or P≠NP. -/
theorem terminal_undertaker_two_boxes (S : ActualSAT) (T : Ladder.{1}) (N
: NormalForm T) (R : OnRow.{1, 1, 0} (actualPNPRow S) T N) (m₀ : R.M) :
    Box1_FormSealed S → (Box2_ZFC_HasNoAct S T N R m₀ → ¬ T.Prov N.V ∧ ¬
    T.Prov N.B) ∧ (PositActualPNP S → (actualPNPRow S).Value
    (actualPNPRow S).compliant) :=
  fun box1 => (fun box2 => two_boxes_together_terminal_ZFC_blocked S T N R
  m₀ box1 box2, fun posit => (the_act_gives_the_value (actualPNPRow S) _
  posit).1)

theorem box1_form_sealed_holds (S : ActualSAT) : Box1_FormSealed S :=
  (airtight (actualPNPRow S),
   fun T N R m p => one_door (actualPNPRow S) T N R p m,
   fun r => r.value_t,
   fun Q hQ x hx q => (hx, fun _ => hQ x hx q))

/-- TWO BOXES HERMETICALLY SEALED TOGETHER: form sealed holds
unconditionally, ZFC has no act is the assumption that makes ZFC boxed
terminally. -/
theorem two_boxes_hermetically_sealed_together (S : ActualSAT) (T :
Ladder.{1}) (N : NormalForm T) (R : OnRow.{1, 1, 0} (actualPNPRow S) T N)
(m₀ : R.M) :
    Box1_FormSealed S ∧ (Box2_ZFC_HasNoAct S T N R m₀ → ¬ T.Prov N.V ∧ ¬
    T.Prov N.B) :=
  (box1_form_sealed_holds S, fun box2 =>
  two_boxes_together_terminal_ZFC_blocked S T N R m₀
  (box1_form_sealed_holds S) box2)


end ZFCBlockHardened

/-! Cones, pinned -/

/-- info: 'ZFCBlockHardened.exit_shared_closed' does not depend on any
axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.exit_shared_closed
/-- info: 'ZFCBlockHardened.exit_record_closed' does not depend on any
axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.exit_record_closed
/-- info: 'ZFCBlockHardened.record_decides_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.record_decides_nothing
/-- info: 'ZFCBlockHardened.exit_bypass_closed' does not depend on any
axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.exit_bypass_closed
/-- info: 'ZFCBlockHardened.no_fourth_kind' does not depend on any axioms
-/
#guard_msgs in #print axioms ZFCBlockHardened.no_fourth_kind
/-- info: 'ZFCBlockHardened.airtight' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.airtight
/-- info: 'ZFCBlockHardened.two_kinds_no_third' does not depend on any
axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.two_kinds_no_third
/-- info: 'ZFCBlockHardened.no_escape' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.no_escape
/-- info: 'ZFCBlockHardened.blocked_of_defiant_model' does not depend on
any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.blocked_of_defiant_model
/-- info: 'ZFCBlockHardened.blocked_iff_defiant_model' does not depend on
any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.blocked_iff_defiant_model
/-- info: 'ZFCBlockHardened.record_bound_ladder_is_blocked' does not
depend on any axioms -/
#guard_msgs in #print axioms
ZFCBlockHardened.record_bound_ladder_is_blocked
/-- info: 'ZFCBlockHardened.proving_ladder_reads_past_the_record' does
not depend on any axioms -/
#guard_msgs in #print axioms
ZFCBlockHardened.proving_ladder_reads_past_the_record
/-- info: 'ZFCBlockHardened.ladder_proof_is_of_both_kinds' does not
depend on any axioms -/
#guard_msgs in #print axioms
ZFCBlockHardened.ladder_proof_is_of_both_kinds
```

```
/-- info: 'ZFCBlockHardened.decided_by_the_models' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.decided_by_the_models
/-- info: 'ZFCBlockHardened.RH.fold_fixed_iff' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms ZFCBlockHardened.RH.fold_fixed_iff
/-- info: 'ZFCBlockHardened.RH.record_on_line' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.RH.record_on_line
/-- info: 'ZFCBlockHardened.RH.W1_closed' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.RH.W1_closed
/-- info: 'ZFCBlockHardened.RH.W1_rh' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.RH.W1_rh
/-- info: 'ZFCBlockHardened.RH.W2_closed' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.RH.W2_closed
/-- info: 'ZFCBlockHardened.RH.W2_not_rh' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.RH.W2_not_rh
/-- info: 'ZFCBlockHardened.RH.same_record' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.RH.same_record
/-- info: 'ZFCBlockHardened.PNP.bnot_ne' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PNP.bnot_ne
/-- info: 'ZFCBlockHardened.PNP.record_closes' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PNP.record_closes
/-- info: 'ZFCBlockHardened.PNP.complement_free' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PNP.complement_free
/-- info: 'ZFCBlockHardened.PNP.closed_world_closes' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PNP.closed_world_closes
/-- info: 'ZFCBlockHardened.PNP.stage_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PNP.stage_escapes
/-- info: 'ZFCBlockHardened.PNP.same_rec' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PNP.same_rec
/-- info: 'ZFCBlockHardened.PC.descent_bound' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PC.descent_bound
/-- info: 'ZFCBlockHardened.PC.witness_gives_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PC.witness_gives_the_act
/-- info: 'ZFCBlockHardened.PC.soliton_rests' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PC.soliton_rests
/-- info: 'ZFCBlockHardened.PC.no_witness_on_the_soliton' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PC.no_witness_on_the_soliton
/-- info: 'ZFCBlockHardened.PC.rec_rests' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PC.rec_rests
/-- info: 'ZFCBlockHardened.PC.steps_stuck' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PC.steps_stuck
/-- info: 'ZFCBlockHardened.PC.stuck_does_not_rest' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PC.stuck_does_not_rest
/-- info: 'ZFCBlockHardened.PC.same_rec' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.PC.same_rec
/-- info: 'ZFCBlockHardened.grand_witness' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.grand_witness
/-- info: 'ZFCBlockHardened.blessing_is_adoption' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.blessing_is_adoption
/-- info: 'ZFCBlockHardened.no_total_wall' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.no_total_wall
/-- info: 'ZFCBlockHardened.total_wall_iff_negation' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.total_wall_iff_negation
/-- info: 'ZFCBlockHardened.irrefutable_is_proved' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ZFCBlockHardened.irrefutable_is_proved
/-- info: 'ZFCBlockHardened.the_zfc_block_hardened' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms ZFCBlockHardened.the_zfc_block_hardened
/-- info: 'ZFCBlockHardened.deny_the_bit_zfc_stays_blocked' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.deny_the_bit_zfc_stays_blocked
/-- info: 'ZFCBlockHardened.reject_our_kind_rejects_zfc' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.reject_our_kind_rejects_zfc
/-- info: 'ZFCBlockHardened.blocked_does_not_deny_the_bit' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.blocked_does_not_deny_the_bit
/-- info: 'ZFCBlockHardened.the_slogan_at_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_slogan_at_theorem_grade
/-- info: 'ZFCBlockHardened.semNormalForm' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.semNormalForm
/-- info: 'ZFCBlockHardened.semOnRow' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.semOnRow
/-- info: 'ZFCBlockHardened.the_ladder_route_is_open' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_ladder_route_is_open
/-- info: 'ZFCBlockHardened.no_block_from_the_row' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.no_block_from_the_row
/-- info: 'ZFCBlockHardened.no_proof_from_the_row' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.no_proof_from_the_row
/-- info: 'ZFCBlockHardened.the_chain_does_not_block' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_chain_does_not_block
/-- info: 'ZFCBlockHardened.ledger_holds' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.ledger_holds
/-- info: 'ZFCBlockHardened.ledger_is_forced' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.ledger_is_forced
/-- info: 'ZFCBlockHardened.the_route_ledger_hardened' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_route_ledger_hardened
/-- info: 'ZFCBlockHardened.the_act_gives_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_act_gives_the_value
/-- info: 'ZFCBlockHardened.every_way_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.every_way_is_the_act
/-- info: 'ZFCBlockHardened.the_act_is_the_weakest' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_act_is_the_weakest
/-- info: 'ZFCBlockHardened.the_act_is_irreducible' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_act_is_irreducible
/-- info: 'ZFCBlockHardened.one_door' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.one_door
/-- info: 'ZFCBlockHardened.the_act_on_the_compliant' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_act_on_the_compliant
/-- info: 'ZFCBlockHardened.the_witness_supplies_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_witness_supplies_the_act
/-- info: 'ZFCBlockHardened.blessing_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.blessing_the_act
/-- info: 'ZFCBlockHardened.our_way_by_theorem' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.our_way_by_theorem
/-- info: 'ZFCBlockHardened.unconditional_visible_block_PNP' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.unconditional_visible_block_PNP
/-- info: 'ZFCBlockHardened.unconditional_visible_block_RH' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.unconditional_visible_block_RH
/-- info: 'ZFCBlockHardened.deny_our_proof_blocks_every_zfc_proof_PNP' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.deny_our_proof_blocks_every_zfc_proof_PNP
/-- info: 'ZFCBlockHardened.deny_our_proof_blocks_every_zfc_proof_RH' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.deny_our_proof_blocks_every_zfc_proof_RH
/-- info: 'ZFCBlockHardened.the_unconditional_block_visible' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.the_unconditional_block_visible
/-- info: 'ZFCBlockHardened.hermetically_sealed' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.hermetically_sealed
/-- info: 'ZFCBlockHardened.final_capstone_hermetically_sealed' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.final_capstone_hermetically_sealed
/-- info: 'ZFCBlockHardened.deny_our_proof_zfc_stays_blocked_hermetically_sealed' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.deny_our_proof_zfc_stays_blocked_hermetically_sealed
/-- info: 'ZFCBlockHardened.Box1_FormSealed' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.Box1_FormSealed
/-- info: 'ZFCBlockHardened.Box2_ZFC_HasNoAct' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.Box2_ZFC_HasNoAct
/-- info: 'ZFCBlockHardened.two_boxes_together_terminal_ZFC_blocked' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.two_boxes_together_terminal_ZFC_blocked
/-- info: 'ZFCBlockHardened.boxed_via_3d_vs_2d' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.boxed_via_3d_vs_2d
/-- info: 'ZFCBlockHardened.terminal_undertaker_two_boxes' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.terminal_undertaker_two_boxes
/-- info: 'ZFCBlockHardened.box1_form_sealed_holds' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.box1_form_sealed_holds
/-- info: 'ZFCBlockHardened.two_boxes_hermetically_sealed_together' does not depend on any axioms -/
#guard_msgs in #print axioms ZFCBlockHardened.two_boxes_hermetically_sealed_together
```